المدرسة الوطنية للعلوم التطبيقية - آسفي
ⵜⵉⵎⵙⵙⵓⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⴰⵡⴰⵍⵜ ⵜⵉⵣⵉⵡⵉⵜ - ⴰⵙⴼⵉ
Ecole Nationale des Sciences Appliquées - Safi

N° d'ordre : ……./...

Université Cadi Ayyad

École Nationale des Sciences Appliquées de Safi

Département Informatique, Réseaux et Télécommunications (IRT)

MÉMOIRE DE PROJET DE FIN D'ETUDES

Pour l'obtention du diplôme :

INGÉNIEUR D'ETAT

En Génie des Télécommunications et Réseaux

MISE EN ŒUVRE DE L'INFRASTRUCTURE RESEAU AS CODE POUR LE DEPLOIEMENT AUTOMATISE

Réalisé par :

MOUHAMMAD Hassani Hamadi

Effectué à :

INEOS TECH FORWARD

Encadré à l'ENSAS par :

Mme. Hind Mestouri

Encadré à INEOS par :

Mr. Moussaàb Ouzid

Soutenue le 26/06/2026 devant le jury :

Mme. Manal Zettam	Professeur à l'ENSA de Safi	- Président
Mr.BENLAMKADEM Abdellatif	Professeur à l'ENSA de Safi	- Rapporteur
Mme. Hind Mestouri	Professeur à l'ENSA de Safi	- Encadrante

Année 2025/2026

Dédicaces

À ma très chère mère,

Pour m'avoir élevé, protégé et porté à bout de bras. Tu t'es battue sur tous les fronts pour que je ne manque de rien. Ce travail est le fruit de tes sacrifices et de ceux qui se sont battus à tes côtés pour ma réussite.

À la mémoire de mon père,

Parti bien trop tôt, mais toujours présent. Merci pour les valeurs d'homme que tu as eu le temps de me transmettre. J'espère que là où tu es, tu es fier de moi. Que ton âme repose en paix.

À la mémoire de ma grand-mère,

Partie si récemment. Ton humour, ton courage face à la maladie et ta tendresse me manquent déjà. Une vraie grand-mère, unique et irremplaçable. Que ton âme repose en paix.

À mes professeurs,

Pour la transmission, la patience et m'avoir donné les clés pour mener à bien ce travail.

À mes proches et à mes amis de toujours,

Ceux qui ont partagé les galères, les rires et qui ont cru en moi à chaque étape de ce parcours. Pas besoin de longs discours pour vous exprimer mon attachement.

Merci d'avoir été là.

Remerciements

*Avant tout, je remercie le **Dieu Tout-Puissant** de m'avoir accordé la santé, la volonté et la persévérance nécessaires pour mener à bien ce projet de fin d'études.*

*Mes sincères remerciements s'adressent à l'ensemble des responsables et corps de l'entreprise d'accueil **INEOS,** en particulier à **Mme. RAJAE EL ABBADI** pour la confiance accordée à ma candidature, leur disponibilité et les conditions favorables qu'ils m'ont offertes tout au long de ce projet.*

*J'exprime ma profonde gratitude à mon **encadrant professionnel, Mr. Moussaàb Ouzid** pour son accompagnement, ses conseils avisés et son soutien tout au long de la réalisation de ce travail.*

*Je tiens également à remercier chaleureusement mon **encadrante pédagogique, Mme. Hind Mestouri** pour son suivi, ses orientations pertinentes et la qualité de son encadrement académique.*

*Mes remerciements s'adressent également à l'ensemble des enseignants de l'établissement **ENSA de SAFI** qui ont contribué à ma formation. Leurs enseignements, leur disponibilité et leur engagement ont constitué un apport précieux dans l'acquisition des connaissances et compétences mobilisées au cours de ce projet.*

*Je remercie enfin les **membres du jury** pour l'intérêt qu'ils portent à ce travail et pour le temps consacré à son évaluation.*

À toutes les personnes qui, de près ou de loin, ont contribué à l'aboutissement de ce projet, j'adresse mes plus sincères remerciements.

Résumé

La gestion des infrastructures réseau modernes est confrontée à plusieurs défis : diversité des équipements, multiplication des changements, risques liés aux interventions manuelles et difficulté à garantir la traçabilité des opérations. Face à ces contraintes, l'automatisation constitue une approche permettant d'améliorer la fiabilité et la cohérence des déploiements.

Ce mémoire présente la conception et la mise en œuvre d'une infrastructure **Network as Code** fondée sur les principes du **NetDevOps**, réalisée dans le cadre d'un projet de fin d'études au sein de la société INEOS à Casablanca. L'objectif principal est de mettre en place une chaîne d'automatisation capable de centraliser les données d'infrastructure, de générer les configurations réseau, de les valider avant déploiement et d'en vérifier la conformité après application.

La solution repose sur une architecture Spine-Leaf simulée sous EVE-NG et intégrant des équipements Cisco IOS, Arista EOS et FortiOS. Elle s'appuie sur **NetBox** comme source de vérité centralisée, **Ansible** pour l'automatisation des déploiements, **Batfish** pour la validation statique des configurations, **pyATS** pour les audits de conformité et **GitHub Actions** pour l'orchestration du pipeline CI/CD. Le projet a été mené selon une démarche Agile inspirée de Scrum.

Les résultats obtenus montrent que l'ensemble du cycle de gestion des configurations peut être automatisé de manière fiable dans un environnement multi-constructeurs. La solution garantit la cohérence entre l'état déclaré et l'état réel du réseau tout en assurant la traçabilité des changements et la reproductibilité des opérations.

Mots-clés : NetDevOps, Network as Code, automatisation réseau, CI/CD, NetBox, Ansible, Batfish, pyATS, multi-constructeurs.

Abstract

Modern network infrastructures face increasing challenges related to equipment diversity, frequent configuration changes, and the limitations of manual administration. These constraints increase the risk of human error, reduce deployment consistency, and make change tracking more difficult. In this context, network automation has become a key approach to improving operational reliability and efficiency.

This thesis presents the design and implementation of a **Network as Code** infrastructure based on **NetDevOps** principles, carried out during a final-year engineering internship at INEOS in Casablanca. The main objective was to develop an automation framework capable of centralizing infrastructure data, generating network configurations, validating them before deployment, and verifying their compliance after implementation.

The proposed solution relies on a Spine-Leaf architecture simulated in EVE-NG and composed of Cisco IOS devices, Arista EOS switches, and FortiOS firewalls. It uses **NetBox** as a centralized source of truth, **Ansible** for configuration deployment and orchestration, **Batfish** for pre-deployment validation, **pyATS** for compliance auditing, and **GitHub Actions** for CI/CD pipeline orchestration. The project was conducted using an Agile approach inspired by Scrum.

The results demonstrate that the entire network configuration lifecycle can be automated in a reliable manner within a multi-vendor environment. The implemented solution ensures consistency between the intended and the actual network state while improving change traceability and deployment reproducibility.

Keywords: NetDevOps, Network as Code, network automation, CI/CD, NetBox, Ansible, Batfish, pyATS, multi-vendor infrastructure.

Liste des abréviations

Accronymes	Signification
ACL	Access Control List
AES	Advanced Encryption Standard (standard de chiffrement avancé)
API	Application Programming Interface
BDR	Backup Designated Router (rôle OSPF)
CI/CD	Continuous Integration / Continuous Deployment
CIDR	Classless Inter-Domain Routing
CLI	Command Line Interface
CMDB	Configuration Management Database
DCIM	Data Center Infrastructure Management
DMZ	Demilitarized Zone (zone démilitarisée réseau)
DNS	Domain Name System
DR	Designated Router (rôle OSPF)
EOS	Extensible Operating System (Arista Networks)
EVE-NG	Emulated Virtual Environment - Next Generation
HA	High Availability (Haute Disponibilité)
HCI	Hyperconverged Infrastructure (Infrastructure Hyperconvergée)
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IA	Intelligence Artificielle
IaC	Infrastructure as Code

IOL	IOS on Linux (Cisco)
IOS	Internetwork Operating System (Cisco Systems)
IPAM	IP Address Management
ISP	Internet Service Provider (Fournisseur d'Accès Internet)
JSON	JavaScript Object Notation
KVM	Kernel-based Virtual Machine
L2	Layer 2 (couche liaison de données)
L3	Layer 3 (couche réseau)
LACP	Link Aggregation Control Protocol (IEEE 802.3ad)
LAG	Link Aggregation Group
LAN	Local Area Network
LXC	Linux Containers
LXD	LXD (hyperviseur de conteneurs, Canonical)
NaC	Network as Code
NAT	Network Address Translation
NetDevOps	Network Development Operations
NetSecOps	Network Security Operations
NGFW	Next Generation Firewall
NTP	Network Time Protocol
OOB	Out-of-Band (accès de gestion hors bande)
OSPF	Open Shortest Path First (protocole de routage à état de liens)
P2P	Point to Point (lien point à point)
PDG	Président Directeur Général

RAM	Random Access Memory
REST	Representational State Transfer
RQ	Redis Queue
SD-WAN	Software-Defined Wide Area Network
SHA	Secure Hash Algorithm
SNMP	Simple Network Management Protocol
SQL	Structured Query Language
SSH	Secure Shell
SSoT	Single Source of Truth (source de vérité unique)
STP	Spanning Tree Protocol
SVN	Subversion (système de contrôle de versions)
URL	Uniform Resource Locator
vCPU	virtual Central Processing Unit
VLAN	Virtual Local Area Network
VM	Virtual Machine
WAN	Wide Area Network
YAML	YAML Ain't Markup Language (format de sérialisation de données)
ZTP	Zero Touch Provisionning

Liste des figures

Figure 1 : Logo d'INEOS TECH FORWARD 17
Figure 2 : Logos des partenaires stratégiques d'INEOS 18
Figure 3 : Organigramme de la direction d'INEOS 19
Figure 4 : La méthodologie Agile (Scrum) 23
Figure 5 : Cycle d'un sprint 24
Figure 6 : Diagram de Gantt 25
Figure 7 : UC-1 : Déclenchement manuel du pipeline 31
Figure 8 : UC-2 : Déclenchement du pipeline via Netbox (Webhook) 32
Figure 9 : UC-3 : Déclenchement du pipeline via git push 33
Figure 10 : Architecture de la topologie 35
Figure 11 : Logo eve-ng 37
Figure 12 : Logo de netbox 37
Figure 13 : Logo d'Ansible 38
Figure 14 : Logo de Batfish 39
Figure 15 : Logo de pyATS 39
Figure 16 : Logo de GitHub Actions 40
Figure 17 : Logo de git 40
Figure 18 : Logo de docker 41
Figure 19 : Pipeline CI/CD d'automatisation réseau 43
Figure 20 : Planification des cinq sprints 44
Figure 21 : Ressources du serveur EVE-NG 47
Figure 22 : Sortie de docker compose ps sur l'état et santé des huit services 49
Figure 23 : Vue d'ensemble de la topologie EVE-NG 50
Figure 24 : Devices NetBox : rôles, plateformes et site 52
Figure 25 : Interfaces NetBox et adresses IP primaires 53
Figure 26 : Vue d'ensemble de la modélisation des Adresses IP 53
Figure 27 : Modélisation des instances ospf 54
Figure 28 : Routes statiques SD-WAN déclarées dans NetBox 55
Figure 29 : Extrait de la sortie de l'etape 6a du pipeline 73
Figure 30 : Extrait de la sortie de l'etape 7a du pipeline 74

Figure 31 : Extrait de l'exécution de l'étape 10a sur le pipeline 75
Figure 32 : Extrait de l'exécution de l'étape 10b dans le pipeleine 76
Figure 33 : Extrait de sortie de l'étape 8a du pipeline 79
Figure 34 : Exécution de l'étape 10c du pipeline 80
Figure 35 : Extrait de l'étape de génération de la fiche de run à l'étape 11 82
Figure 36 : Vue du pipeline #88 dans GitHub Actions 84
Figure 37 : Fichier Dockerfile-Plugins 89
Figure 38 : Ficher docker-compose.override.yml 90
Figure 39 : Configuration de ISP-1 91

Liste des tableaux

Tableau 1 : Comparaison entre approche traditionnelle et approche NetDevOps 20
Tableau 2 : Acteurs du projet 22
Tableau 3 : Catalogue des besoins fonctionnels du projet d'automatisation reseau 29
Tableau 4 : Catalogue des besoins non fonctionnels..... 33
Tableau 5 : Présentation du backlog des cinq sprints..... 46
Tableau 6 : Plan d'adressage complet de l'environnement de laboratoire 47
Tableau 7 : Inventaire des équipements simules dans EVE-NG..... 51
Tableau 8 : Details des liens entre les routeurs ISP et le pare-feu SD-WAN_NGFW..... 51
Tableau 9 : Interfaces du nœud d'automatisation Linux 51
Tableau 10 : Details des 10 etapes du pipeline CI/CD..... 58
Tableau 11 : Sequencement des six etapes de la phase DEPLOY 70

Table des matières

Dédicaces ii
Remerciements iii
Résumé iv
Abstract v
Liste des abréviations vi
Liste des figures ix
Liste des tableaux xi
Table des matières xii
Introduction générale 15
Chapitre 1 : Contexte général du projet 16
1. Introduction 17
2. Présentation de l'organisme d'accueil 17
2.1 Présentation générale 17
2.2 Services et domaines d'activité 17
2.3 Partenariats technologiques 18
3. Contexte du projet 19
4. Problématique du projet 21
5. Objectifs du projet 21
6. Conduite du projet 22
6.1 Méthodologie du projet 22
6.2 Planification du projet : 24
7. Conclusion 26
Chapitre 2 : Étude de conception 27
1. Introduction 28

2. Définitions des concepts clés du projet 28
3. Etude fonctionnelle 29
3.1 Besoins fonctionnels 29
3.2 Scénarios d'utilisation 30
3.3 Besoins non fonctionnels 33
4. Infrastructure réseau cible 34
4.1 Architecture physique 34
4.2 Planification du trafic 36
5. Choix des outils utilisés 37
6. Architecture de la solution 41
7. Planification la mise en œuvre 44
8. Conclusion 44
Chapitre 3 : Réalisation et tests 45
1. Introduction 46
2. Sprint 1 : Modélisation des données et fondation de la source de vérité 46
2.1 Serveur hôte et plans réseau 47
2.2 Infrastructure Docker 48
2.3 Topologie EVE-NG et connectivité 49
2.4 Modélisation des équipements 52
2.5 Modélisation des interfaces et de l'adressage 53
2.6 Modélisation du routage 54
3. Sprint 2 : Génération des configurations et validation statique 55
3.1 Architecture du pipeline CI/CD 56
3.2 Synchronisation avec la source de vérité 59
3.3 Génération des configurations 62
3.4 Validation statique 64
4. Sprint 3 : Déploiement réseau et validation opérationnelle 66

4.1 Collecte de la baseline 66
4.2 Séquencement et conditions d'exécution 70
4.3 Déploiement Cisco IOS et Arista EOS 71
4.4 Conformité post-déploiement Cisco et Arista 74
5. Sprint 4 : Sécurisation périmétrique et validation de l'idempotence 77
5.1 Déploiement FortiOS 77
5.2 Patterns d'idempotence et gestion des ressources 77
5.3 Résultat du déploiement FortiOS 78
5.4 Conformité post-déploiement FortiOS 79
5.5 Idempotence globale 80
6. Sprint 5 : Audit, conformité et traçabilité 81
6.1 Audit post-déploiement multi-constructeurs 81
6.2 Archivage et nettoyage 81
7. Contraintes techniques et ajustements 85
8. Conclusion 86
Conclusion générale et perspectives d'évolution 87
ANNEXES 89
ANNEXE A : Infrastructure de simulation et configuration Docker 89
A.1 Personnalisation de l'image NetBox 89
A.2 Configuration de référence (routeur ISP-1) 90
ANNEXE B : Extraits sélectionnés des scripts d'automatisation 91
B.1 Structure du workflow CI/CD 92
Références 97

Introduction générale

Les infrastructures réseau constituent aujourd'hui l'épine dorsale des systèmes d'information d'entreprise. Leur gestion fait face à des contraintes de plus en plus fortes : hétérogénéité des équipements, accélération des changements, risques liés aux interventions manuelles et nécessité d'une traçabilité structurée. Dans ce contexte, les méthodes d'administration traditionnelles montrent rapidement leurs limites.

Ce mémoire s'inscrit dans cette problématique et présente la conception ainsi que la mise en œuvre d'une chaîne d'automatisation réseau réalisée lors d'un stage de fin d'études au sein de la société INEOS. Le projet repose sur le paradigme NetDevOps, qui applique au domaine réseau des principes issus du développement logiciel : infrastructure décrite comme du code, source de vérité centralisée, validation systématique et déploiement piloté par pipeline CI/CD.

L'objectif est de mettre en place un mécanisme capable de gérer des changements de configuration sur un environnement multi-constructeurs de manière automatisée, auditable et reproductible, tout en garantissant la cohérence entre l'état déclaré dans la source de vérité et l'état réel observé sur le réseau. Le projet s'appuie pour cela sur une démarche itérative organisée en cinq sprints, depuis la modélisation des données jusqu'à l'audit post-déploiement.

Le mémoire est structuré en trois chapitres. Le chapitre 1 présente le contexte général du projet, la problématique, les objectifs et la démarche méthodologique. Le chapitre 2 traite de l'analyse des besoins, des choix technologiques et de l'architecture de la solution. Le chapitre 3 décrit la mise en œuvre technique de la chaîne d'automatisation, depuis la source de vérité jusqu'à l'audit de conformité.

Chapitre 1 : Contexte général du projet

1. Introduction

Ce chapitre pose le cadre général du projet. Il présente d'abord l'organisme d'accueil dans lequel le stage a été réalisé, décrit le contexte opérationnel qui en a justifié la démarche, analyse les limites des pratiques traditionnelles de gestion réseau, puis formule la problématique centrale et les objectifs poursuivis. Il délimite enfin le périmètre du travail réalisé et présente la démarche méthodologique adoptée.

2. Présentation de l'organisme d'accueil

2.1 Présentation générale

INEOS (Tech Forward) est une société marocaine de services numériques implantée à Casablanca (Lotissement Florida Center Park). Forte d'une équipe de 80 consultants certifiés, elle se positionne comme spécialiste du conseil et de l'intégration de solutions réseau, cloud et sécurité, au service de la transformation digitale des entreprises du Maroc et d'Afrique de l'Ouest. La direction est assurée par M. Reda Bakkali, PDG fort de plus de 25 ans d'expérience dans les technologies de l'information.

Figure 1 : Logo d'INEOS TECH FORWARD

2.2 Services et domaines d'activité

L'offre d'INEOS s'articule en quatre lignes de service : audit et conseil, gestion de projet, intégration et maintenance & support. Ces prestations couvrent trois domaines techniques :

- **Infrastructure cloud** : virtualisation, stockage haute disponibilité, sauvegarde
- **Réseaux télécoms** : LAN, WAN, SD-WAN, commutation, routage, communication unifiée

- **Cybersécurité** : sécurité périmétrique, services managés, protection sur l'ensemble du cycle de vie de la menace

2.3 Partenariats technologiques

INEOS s'appuie sur sept partenaires technologiques stratégiques : Dell EMC, Cisco, Cisco Meraki, VMware, SolarWinds, Fortinet, PaloAlto, Ruckus et Dynatrace. Ces partenariats permettent de délivrer des solutions intégrées best-of-breed à forte valeur ajoutée.



Figure 2 : Logos des partenaires stratégiques d'INEOS

2.4 Références clients

Les références d'INEOS couvrent des projets réalisés pour de grands groupes financiers et bancaires marocains :

- Refonte complète d'infrastructures réseau et sécurité pour un établissement bancaire de premier plan
- Déploiement de plateformes de production et de secours pour des sociétés de financement
- Mise en place d'infrastructures hyperconvergées (HCI) répondant aux exigences de haute disponibilité

Ces interventions confirment le positionnement d'INEOS en tant qu'intégrateur de référence sur des environnements à forte criticité. Par ailleurs, la société développe activement deux axes stratégiques en adéquation directe avec les objectifs du présent stage :

- Des offres de sécurité managée portées par sa filiale INEOS Cyberforce

- L'automatisation des infrastructures réseau pilotée par pipeline CI/CD

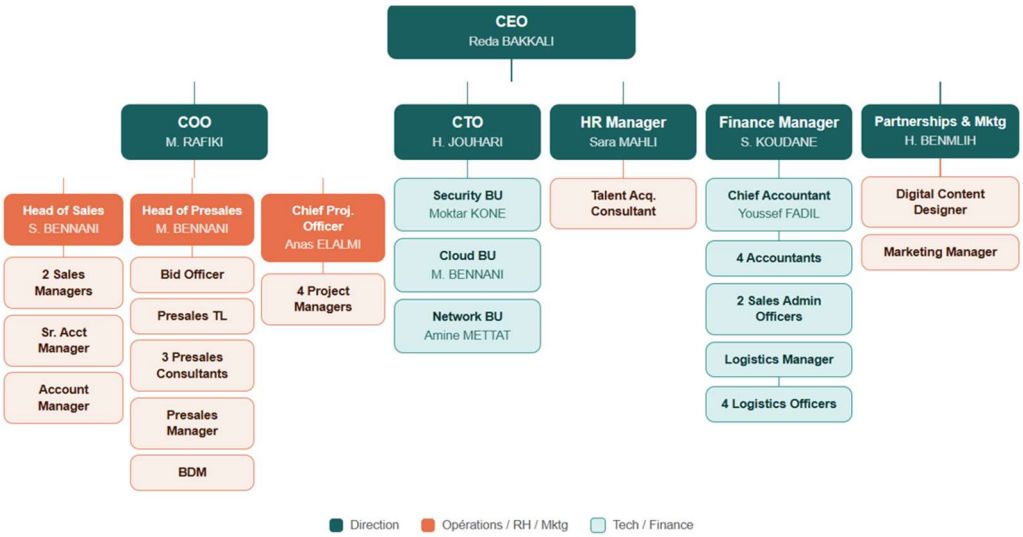


Figure 3 : Organigramme de la direction d'INEOS

Pour plus d'informations sur l'entreprise, voir (INEOS Tech Forward, s.d.).

3. Contexte du projet

En tant qu'intégrateur de solutions réseau, INEOS intervient quotidiennement sur les infrastructures de ses clients dans des environnements multi-constructeurs à forte criticité. La densification des parcs d'équipements, l'hétérogénéité des plateformes déployées, l'accélération des demandes de changement et les obligations croissantes de traçabilité imposées par les directions des systèmes d'information font désormais partie du cadre opérationnel standard de ses missions. Dans ce contexte, les pratiques d'administration manuelle via l'interface de ligne de commande (CLI), encore largement répandues, montrent des limites structurelles que ni l'intégrateur ni ses clients ne peuvent continuer à ignorer..

Ces limites se manifestent sur quatre plans :

- **Risque d'erreur :** toute intervention CLI directe expose l'infrastructure à des erreurs de saisie dont les conséquences peuvent être difficiles à diagnostiquer et à corriger.
- **Absence de traçabilité :** sans versionnement structuré, il est impossible de reconstituer l'historique des modifications ni de rétablir un état antérieur connu.

- **Non-reproductibilité :** deux opérateurs appliquant le même changement peuvent aboutir à des états de configuration différents, générant progressivement des dérives entre équipements.
- **Coût opérationnel :** l'exécution manuelle de changements sur un parc étendu mobilise du temps et des ressources pour des tâches à faible valeur ajoutée.

En réponse à ces limites, INEOS a initié ce projet de fin d'études, avec pour ambition de mettre en place une chaîne d'automatisation réseau fondée sur le paradigme NetDevOp, qui transpose au domaine de l'infrastructure réseau les principes du génie logiciel. L'infrastructure y est décrite comme du code, versionnée dans une source de vérité unique (SSoT) et déployée via un pipeline d'intégration et de déploiement continus (CI/CD). Deux principes en sont fondateurs : l'idempotence, toute opération peut être rejouée sans effet de bord, et la validation systématique avant application.

Tableau 1 : Comparaison entre approche traditionnelle et approche NetDevOps

Critère	**Approche traditionnelle**	**Approche NetDevOps**
Production du changement	Saisie CLI manuelle, non versionnée	Configurations dérivées d'une source de vérité, versionnées
Validation	Contrôle visuel ponctuel, non systématique	Validation automatique avant toute application réseau
Traçabilité	Partielle ou absente	Historique complet de chaque changement
Reproductibilité	Dépendante de l'opérateur	Exécution idempotente, résultat identique à chaque exécution
Délai de livraison	Variable, souvent élevé	Réduit grâce à l'orchestration automatisée

Détection d'anomalie	Post-impact, souvent tardive	Blocage préventif avant application et vérification post-déploiement

C'est ce paradigme que le présent projet met en œuvre. L'architecture retenue et les outils utilisés sont présentés au chapitre 2.

4. Problématique du projet

La gestion des infrastructures réseau modernes s'appuie sur des environnements hétérogènes composés d'équipements et de technologies aux modes d'administration différents. Dans un tel contexte, l'administration manuelle favorise les erreurs opérationnelles, les dérives de configuration entre l'état attendu et l'état réel du réseau, ainsi que les difficultés de traçabilité et de reproductibilité des changements. L'absence de mécanismes centralisés de gestion et de versionnement complique également le maintien d'une infrastructure cohérente et conforme aux exigences d'exploitation.

Face à ces contraintes, la problématique de ce projet consiste à déterminer comment mettre en œuvre une chaîne d'automatisation réseau capable de centraliser la définition de l'état désiré, d'automatiser l'application des changements et de garantir la cohérence, la fiabilité ainsi que la traçabilité des opérations au sein d'une infrastructure multi-constructeurs.

5. Objectifs du projet

Afin de répondre à cette problématique, le projet vise à :

- Mettre en place une source de vérité centralisée décrivant l'état désiré de l'infrastructure.
- Automatiser la génération et le déploiement des configurations réseau sur des équipements multi-constructeurs.
- Valider les configurations avant leur mise en production afin de prévenir les erreurs de déploiement.

- Vérifier la conformité du réseau avant et après chaque changement.
- Assurer la traçabilité complète des opérations réalisées et des résultats obtenus.

6. Conduite du projet

6.1 Méthodologie du projet

Le choix entre un modèle de processus de développement et un autre, dépend de la nature du projet et de sa taille. Lorsqu'il s'agit d'un projet où les données ne sont pas réunies dès le départ, où les besoins sont incomplets, il est recommandé de s'orienter vers une méthode itérative ou orientée prototypes.

Le projet a été conduit selon une adaptation d'une des méthodologies agiles : **Scrum**. La nature exploratoire du travail, avec des contraintes techniques non entièrement anticipables et des composants fortement interdépendants, justifiait une approche itérative permettant de valider chaque incrément avant d'engager le suivant. Les sprint reviews ont coïncidé avec les points d'avancement réguliers avec M. Moussaàb Ouzid, au cours desquels chaque livrable était présenté et validé.

Tableau 2 : Acteurs du projet

Rôle	**Identité**	**Responsabilités**
Binôme	Mouhammad Hassaani Hamadi (Moi)	Conception et développement de la chaîne d'automatisation. Le rôle de Scrum Master a été assumé conjointement par les deux membres, conformément au principe d'auto-organisation caractéristique des petites équipes Agile
	Mohammed Amine Fahmaoui	
Encadrant	Mr. Moussaàb Ouzid	*Product Owner* : définition de l'architecture réseau cible, validation des choix topologiques et arbitrage technique. Son périmètre se limite à la dimension réseau et aux conseils d'encadrement ; la conception de l'automatisation relève du binôme.

Choix et adaptation de Scrum

La figure suivante explique le fonctionnement de cette méthodologie :

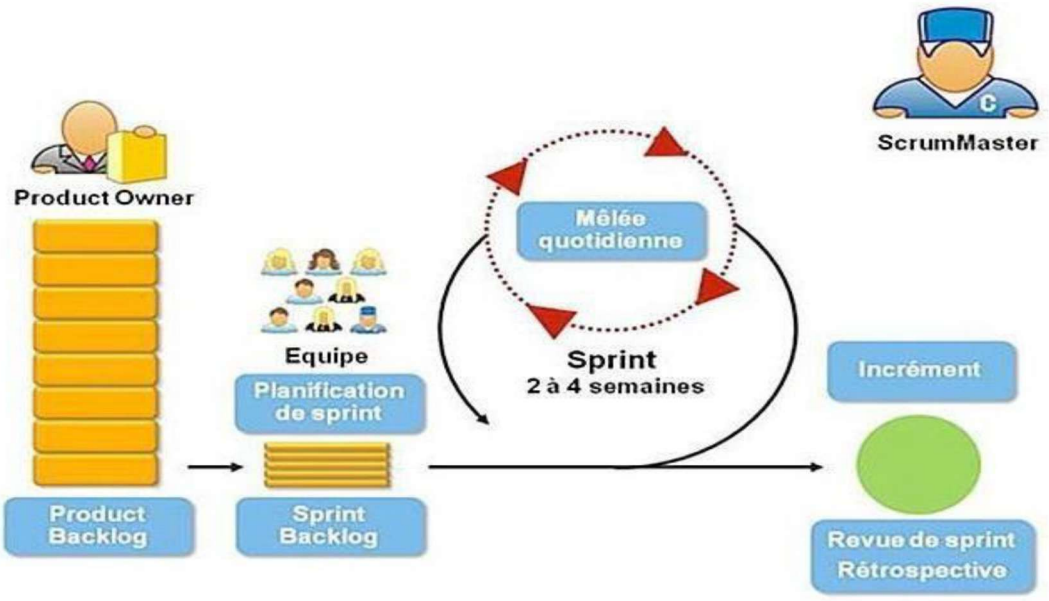


Figure 4 : La méthodologie Agile (Scrum)

Comme le représente ce schéma, avec SCRUM le projet est décomposé en incréments qui traitent les fonctionnalités relevées dans le Backlog une à une d'une manière itérative jusqu'à l'aboutissement du produit final. Grâce à une mêlée quotidienne nous pouvons évaluer le progrès du travail accompli la veille et nous focaliser sur les tâches qui restent à accomplir.

Le choix de la méthodologie Scrum pour le pilotage du projet repose sur les avantages suivants :

- ✓ Une grande flexibilité dans la gestion et l'organisation des travaux.
- ✓ Une forte capacité d'adaptation aux changements grâce à des itérations courtes et régulières.
- ✓ Une meilleure réactivité face aux nouvelles exigences et aux retours des parties prenantes.
- ✓ Une approche pragmatique, conciliant les dimensions théoriques et opérationnelles du projet.
- ✓ Une méthodologie proche des conditions réelles de développement, favorisant la collaboration et l'amélioration continue.

Le projet s'est ainsi déroulé en cinq sprints, chacun ciblant un incrément fonctionnel défini.

La démarche itérative a permis d'absorber les ajustements identifiés en cours de réalisation sans remise en cause de l'architecture globale.

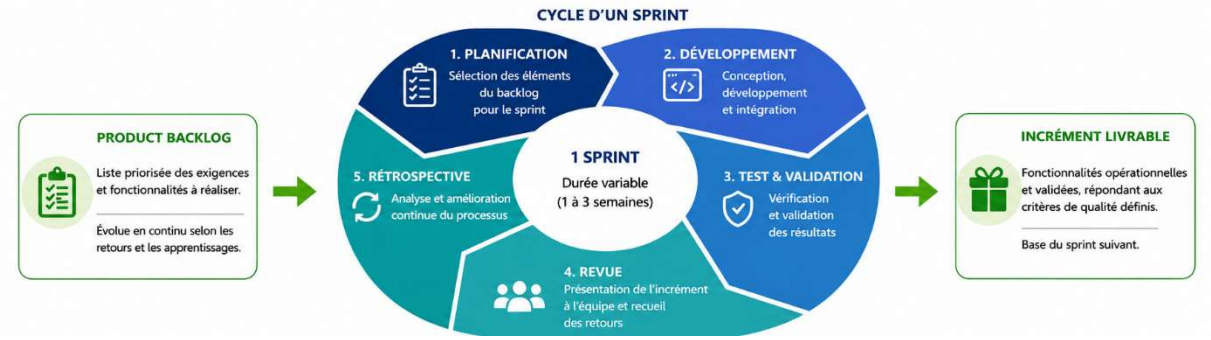


Figure 5 : Cycle d'un sprint

6.2 Planification du projet :

Avant de se lancer dans la réalisation du projet, il est nécessaire de prendre le temps de découper celui en sprint, afin de planifier l'exécution de ces sprints et le temps alloué pour chacun.

Le diagramme de Gantt est un outil permettant de modéliser la planification des tâches nécessaires à la réalisation de notre projet.

La figure suivante représente le planning prévisionnel de notre projet détaillé :

N°	Nom de la tâche	Durée	Début	Fin
1	Étude du sujet et analyse des besoins	10 jours	Mer 18/02/26	Mar 03/03/26
2	Conception de l'architecture et choix technologiques	10 jours	Mer 04/03/26	Mar 17/03/26
3	Conception du Lab	10 jours	Mer 18/03/2(	Mar 31/03/2(
4	Sprint 1 : Modélisation SSoT	15 jours	Mer 01/04/26	Mar 21/04/26
5	Sprint 2 : construction des templates JInja2 et	15 jours	Mer 22/04/26	Mar 12/05/26
6	Sprint 3 : Construction des scripts de déploiement Cisco	8 jours	Mer 13/05/26	Ven 22/05/26
7	Sprint 4 : Sécurisation	5 jours	Lun 25/05/26	Ven 29/05/26
8	Sprint 5 : Construction des mécanismes d'audit	5 jours	Lun 01/06/26	Ven 05/06/26

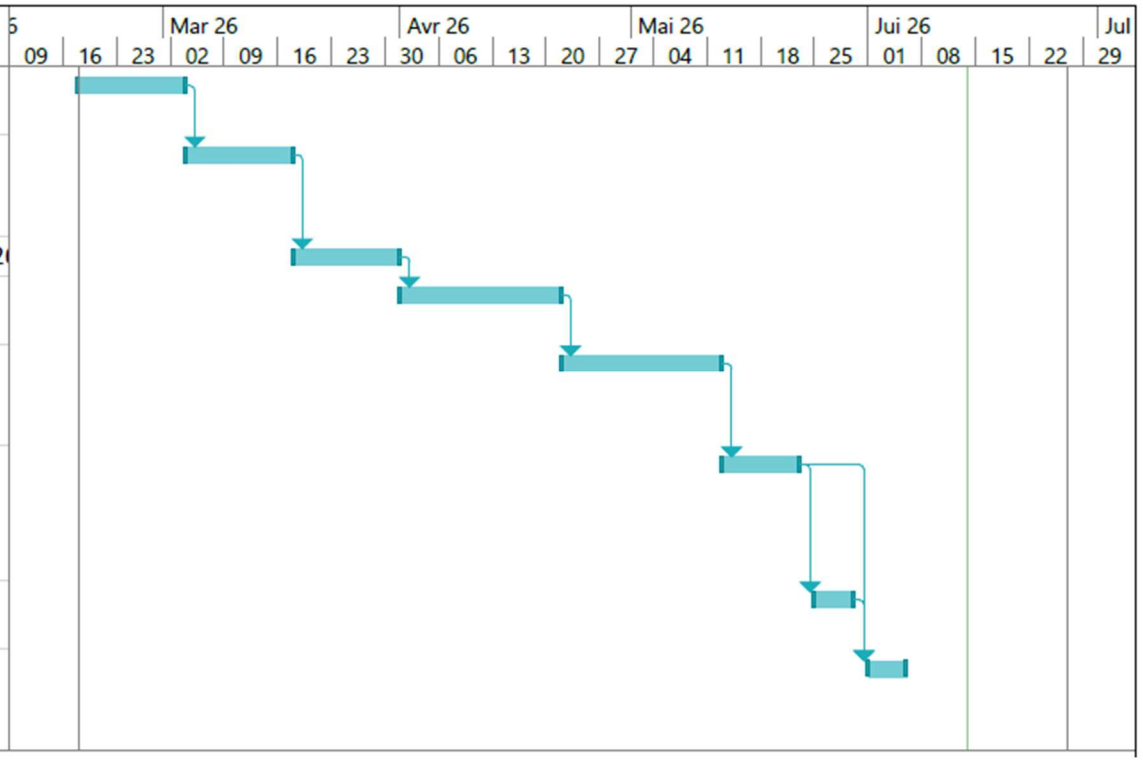


Figure 6 : Diagram de Gantt

7. Conclusion

Ce chapitre a établi le cadre du projet en présentant l'organisme d'accueil INEOS, en analysant les limites structurelles de l'administration réseau manuelle et en formalisant la problématique et les objectifs. La démarche Agile Scrum retenue et la planification en cinq sprints constituent le fil conducteur de l'ensemble du travail. Le chapitre suivant entre dans le vif du sujet en abordant l'analyse des besoins fonctionnels et non fonctionnels, les choix technologiques retenus ainsi que la conception de l'architecture de la solution.

Chapitre 2 : Étude de conception

1. Introduction

Ce chapitre présente l'étude de conception de la chaîne d'automatisation réseau en établissant les fondations conceptuelles et architecturales du projet. À partir des définitions des concepts et de l'analyse des besoins, il en dérive les exigences fonctionnelles, justifie les choix technologiques retenus, et aboutit à l'architecture cible et à la planification en cinq sprints qui structurent le chapitre suivant.

2. Définitions des concepts clés du projet

Avant d'aborder l'infrastructure et les besoins, il est nécessaire d'aborder les notions fondamentales sur lesquelles repose la solution. Ces concepts constituent le vocabulaire commun du projet et guident l'ensemble des choix architecturaux.

Network as Code (NaC)

Le *Network as Code* constitue une extension naturelle du paradigme *Infrastructure as Code* appliquée au domaine des réseaux. Il consiste à représenter l'ensemble des configurations réseau sous forme de fichiers déclaratifs versionnés, manipulés comme du code source.

Dans cette approche, les configurations :

- sont soumises à des processus de revue,
- sont historisées dans un système de gestion de versions,
- et sont déployées via des pipelines automatisés.

Ainsi, le NaC remplace la configuration manuelle interactive par une gestion reproductible, traçable et auditable de l'infrastructure.

Infrastructure as Code (IaC)

Plus général que le NaC, l'IaC désigne la pratique de gérer et de provisionner toute infrastructure serveurs, réseau, stockage par des fichiers de configuration plutôt que par des processus manuels. L'IaC garantit la cohérence des environnements, facilite la reproductibilité et permet l'intégration des infrastructures dans les cycles de livraison logicielle.

Source de Vérité Unique (SSoT)

La SSoT est un référentiel centralisé qui fait autorité sur l'état désiré de l'ensemble de l'infrastructure. Toute configuration déployée est dérivée de ce référentiel ; tout écart entre la SSoT et l'état réel d'un équipement constitue une dérive à corriger. La SSoT élimine les désaccords entre systèmes partiels de documentation et garantit la cohérence globale.

NetDecOps

Le NetDevOps applique aux infrastructures réseau les principes issus du DevOps, notamment l'automatisation, le versionnement, l'intégration continue et l'infrastructure as code. Cette approche permet de gérer les configurations réseau à travers des pipelines automatisés garantissant la cohérence, l'idempotence, la traçabilité et la reproductibilité des changements. Elle favorise également une collaboration plus efficace entre les équipes et une réduction des interventions manuelles tout au long du cycle de vie des configurations.

Pipeline CI/CD réseau

Un pipeline d'Intégration Continue / Déploiement Continu (CI/CD) appliqué au réseau est une chaîne d'étapes automatisées déclenchées par un événement modification de la SSoT, déclenchement manuel et qui orchestre séquentiellement la génération, la validation, le déploiement et l'audit des configurations réseau. Chaque étape est une porte de qualité : un échec bloque la suite et préserve le réseau dans son état antérieur.

3. Etude fonctionnelle

3.1 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit savoir faire :

Tableau 3 : Catalogue des besoins fonctionnels du projet d'automatisation reseau

Intitulé	Description
Synchronisation SSoT	Extraction automatique des données d'infrastructure depuis le référentiel centralisé et structuration en variables exploitables, sans intervention manuelle.

Génération de configurations	Production automatique des configurations réseau adaptées à chaque famille d'équipements, en cohérence avec l'état déclaré dans la SSoT.
Validation pré-déploiement	Vérification statique des configurations générées avant déploiement, avec blocage en cas d'erreur détectée.
Déploiement automatisé	Application des configurations validées de manière séquentielle, idempotente et sans intervention manuelle, sur l'ensemble des plateformes.
Sécurisation périmétrique	Intégration de la configuration et du durcissement des pare-feux FortiGate dans le pipeline d'automatisation.
Audit opérationnel	Capture et comparaison de l'état du réseau avant et après déploiement, avec génération d'un rapport de conformité.
Enforcement SSoT	Détection et correction automatique des dérives entre l'état réel des équipements et l'état déclaré dans la SSoT.

3.2 Scénarios d'utilisation

Afin de répondre aux contraintes précédemment identifiées, la solution s'appuie sur plusieurs scénarios de fonctionnement permettant de couvrir les principaux cas d'utilisation rencontrés lors de l'exploitation du réseau. Ces scénarios illustrent les différents mécanismes de déclenchement du pipeline d'automatisation ainsi que les interactions entre les composants de la plateforme.

Le diagramme de cas d'utilisation est un diagramme UML fournissant une vision statique et globale des comportements fonctionnels d'un système logiciel. Il est composé des acteurs externes et de cas d'utilisation.

- **Les acteurs :** toute entité externe qui interagit avec le système.
- **Un cas d'utilisation :** représente un ensemble de séquences d'actions réalisées par le système et produisant un résultat observable intéressant pour un acteur particulier. Il permet de décrire ce que le futur système devra faire, sans spécifier comment il le fera.

✓ Scénario 1 : déclenchement manuel

Ce scénario correspond à une exécution volontaire du pipeline par un administrateur ou un ingénieur NetDevOps depuis l'interface GitHub Actions. Il permet de lancer l'ensemble des

étapes de génération, de validation et de déploiement indépendamment de toute modification préalable de l'infrastructure ou du dépôt Git.

Cette fonctionnalité est notamment utilisée pour les opérations de maintenance, les tests de validation ou les redéploiements planifiés.

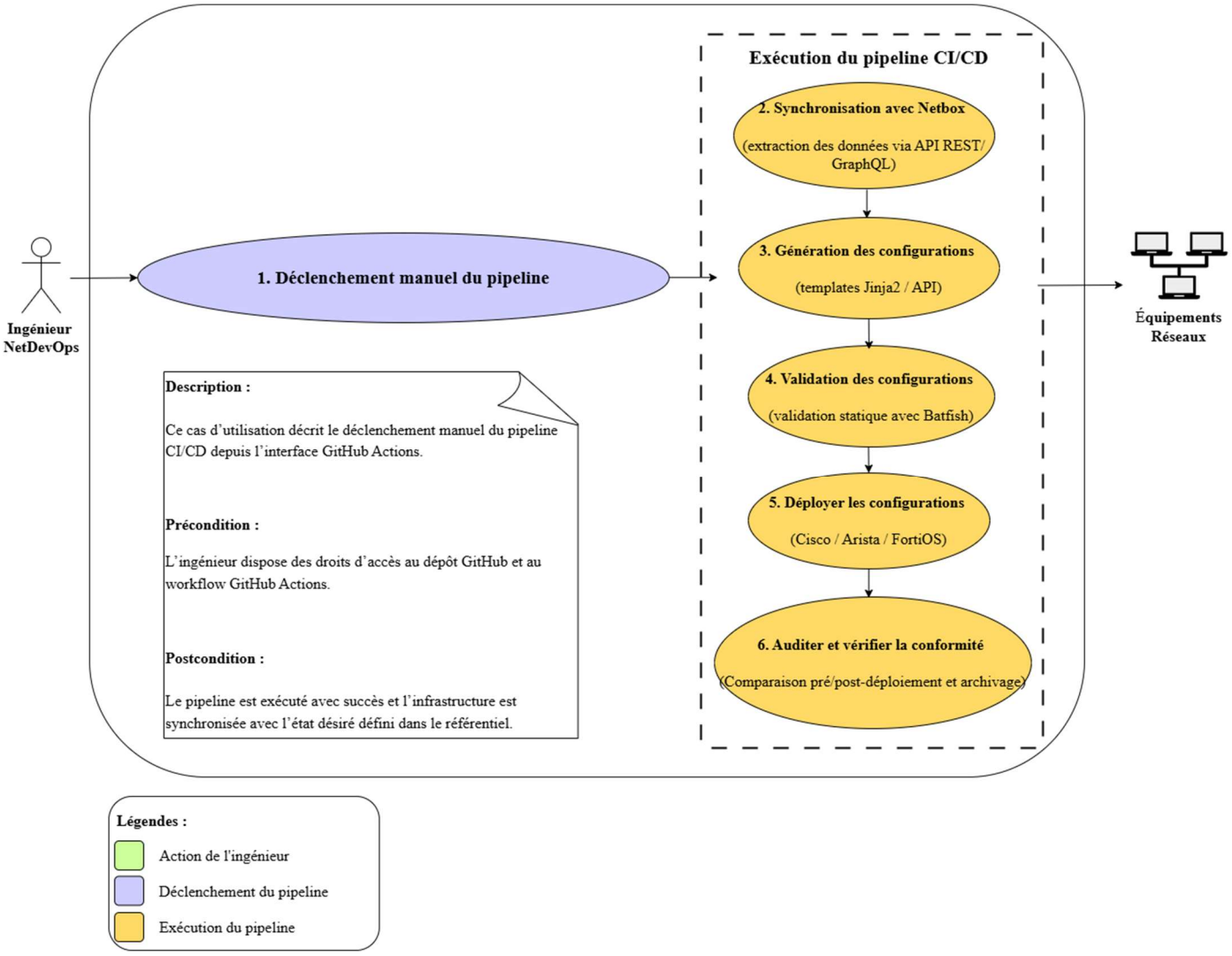


Figure 7 : UC-1 : Déclenchement manuel du pipeline

✓ Scénario 2 : déclenchement depuis la SSoT

Lorsqu'une modification est effectuée dans le référentiel, un mécanisme de déclenchement manuel via Webhook initie la synchronisation des données d'infrastructure. Les informations sont alors extraites, transformées en variables exploitables, puis intégrées au pipeline afin de générer des configurations cohérentes avec l'état déclaré dans la Source of Truth.

Ce scénario garantit que les configurations produites reflètent en permanence les données de référence centralisées.

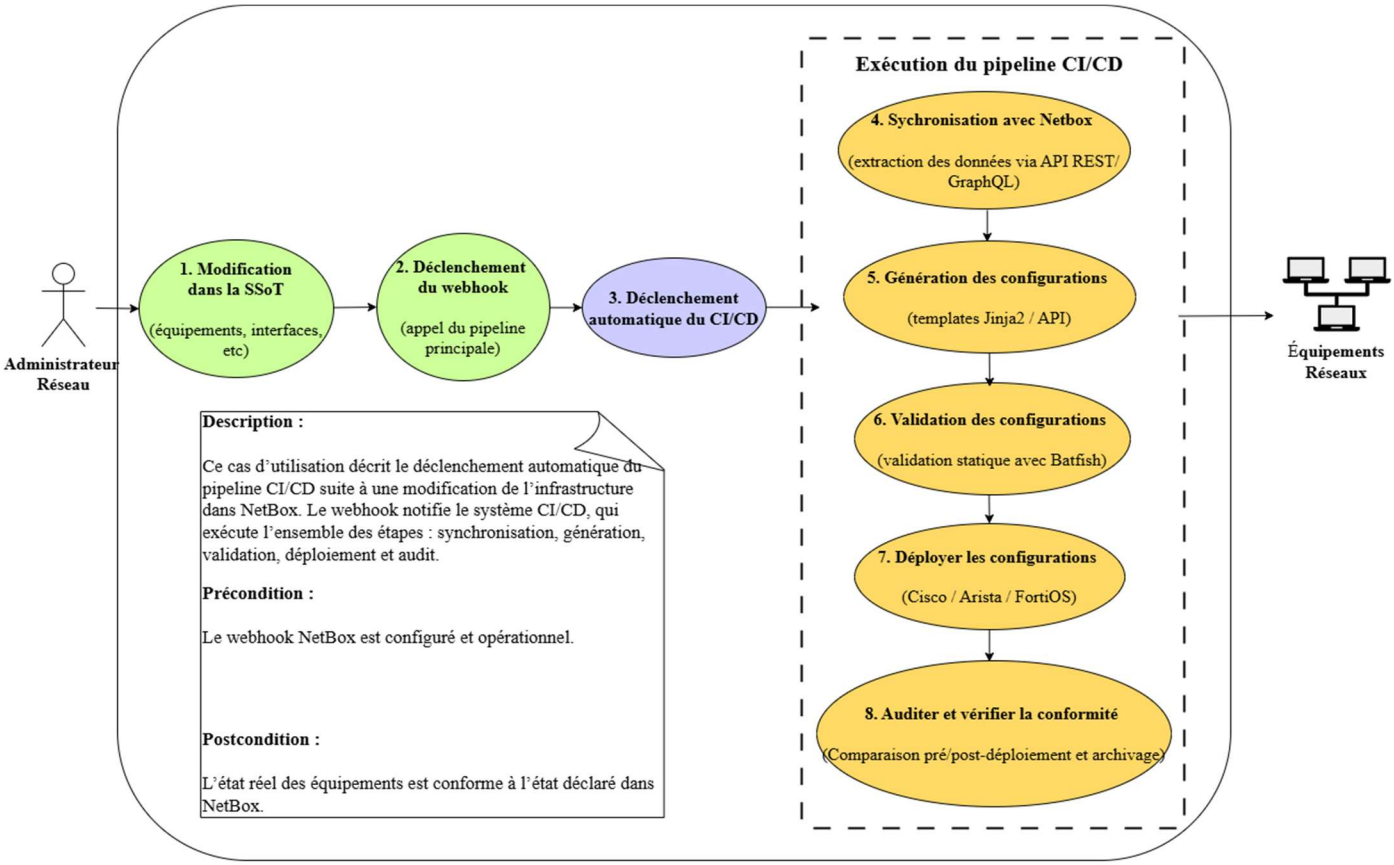


Figure 8 : UC-2 : Déclenchement du pipeline via Netbox (Webhook)

✓ Scénario 3 : déclenchement depuis un versionnement

Lorsqu'un ingénieur effectue une modification du code d'automatisation et la publie dans le dépôt Git, le pipeline CI/CD est automatiquement déclenché.

L'intégration du système de gestion de versions assure une traçabilité complète des évolutions du projet, facilite les retours en arrière et garantit la reproductibilité des déploiements.

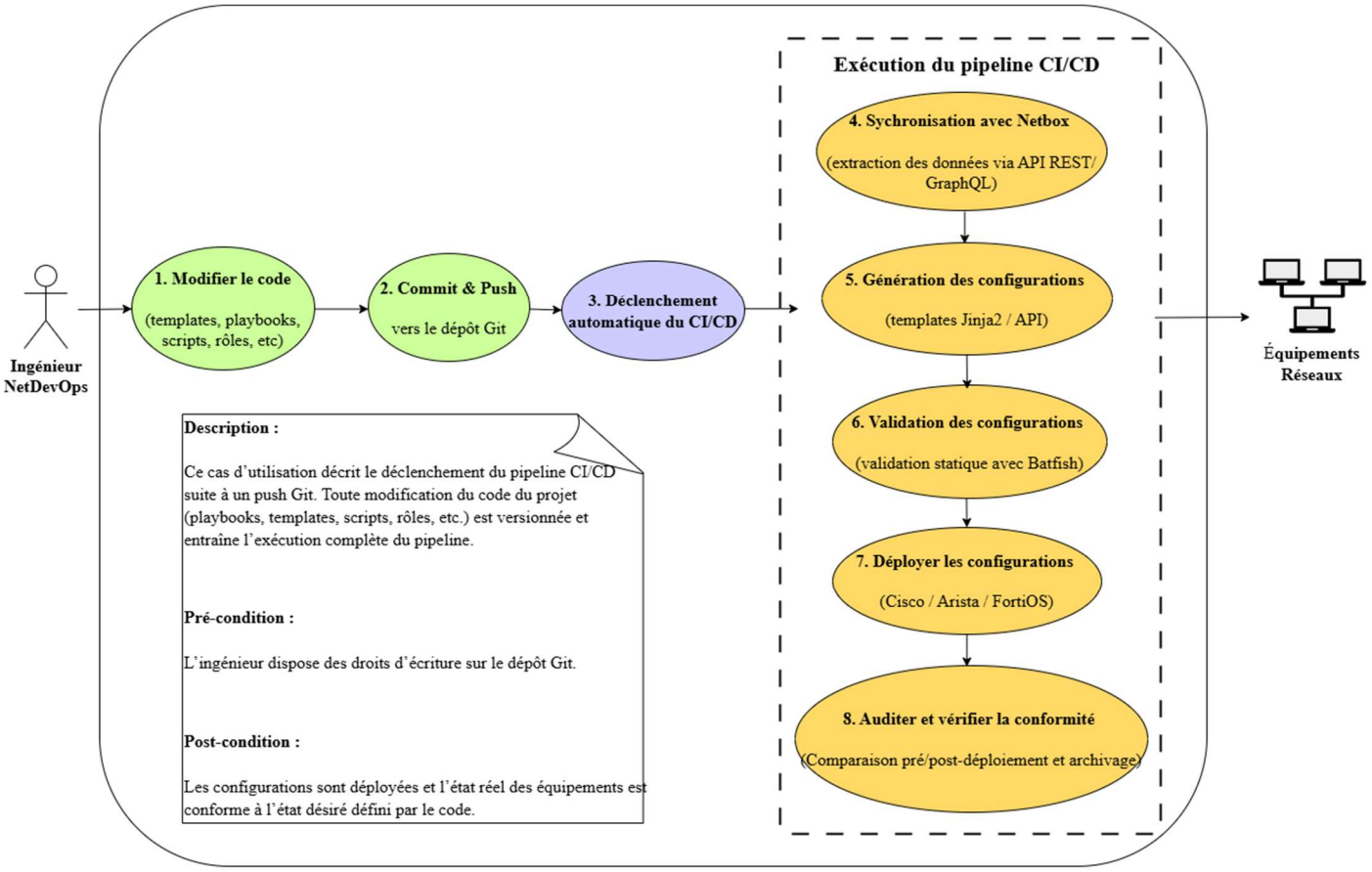


Figure 9 : UC-3 : Déclenchement du pipeline via git push

3.3 Besoins non fonctionnels

Les exigences non fonctionnelles garantissant la qualité, la robustesse et l'exploitabilité du système sont présentées dans le tableau suivant :

Tableau 4 : Catalogue des besoins non fonctionnels

Intitulé	Description
Performance	Le système doit exécuter les processus de génération, validation et déploiement dans des délais compatibles avec un usage opérationnel.
Fiabilité	Les opérations doivent être robustes, avec gestion des erreurs et garantie de cohérence des configurations produites.
Idempotence	Les déploiements doivent être reproductibles sans effet de bord en cas d'exécutions multiples.

Maintenabilité	L'architecture et le code doivent être structurés, modulaires et facilement évolutifs.
Scalabilité	Le système doit pouvoir s'adapter à une augmentation du nombre d'équipements et de configurations.
Traçabilité	Toutes les actions (génération, validation, déploiement) doivent être journalisées et auditables.
Sécurité	Les échanges et accès aux équipements doivent être sécurisés (authentification, chiffrement, contrôle d'accès).
Portabilité	La solution doit être indépendante de l'environnement et facilement déployable sur différents systèmes.

4. Infrastructure réseau cible

4.1 Architecture physique

L'infrastructure étudiée repose sur une architecture moderne inspirée des data centers, conçue pour garantir à la fois performance, résilience et évolutivité.

L'infrastructure réseau cible est organisée selon une topologie **Spine-Leaf**, modèle dominant dans les centres de données modernes pour sa prévisibilité de latence, sa haute disponibilité et sa capacité à évoluer horizontalement.

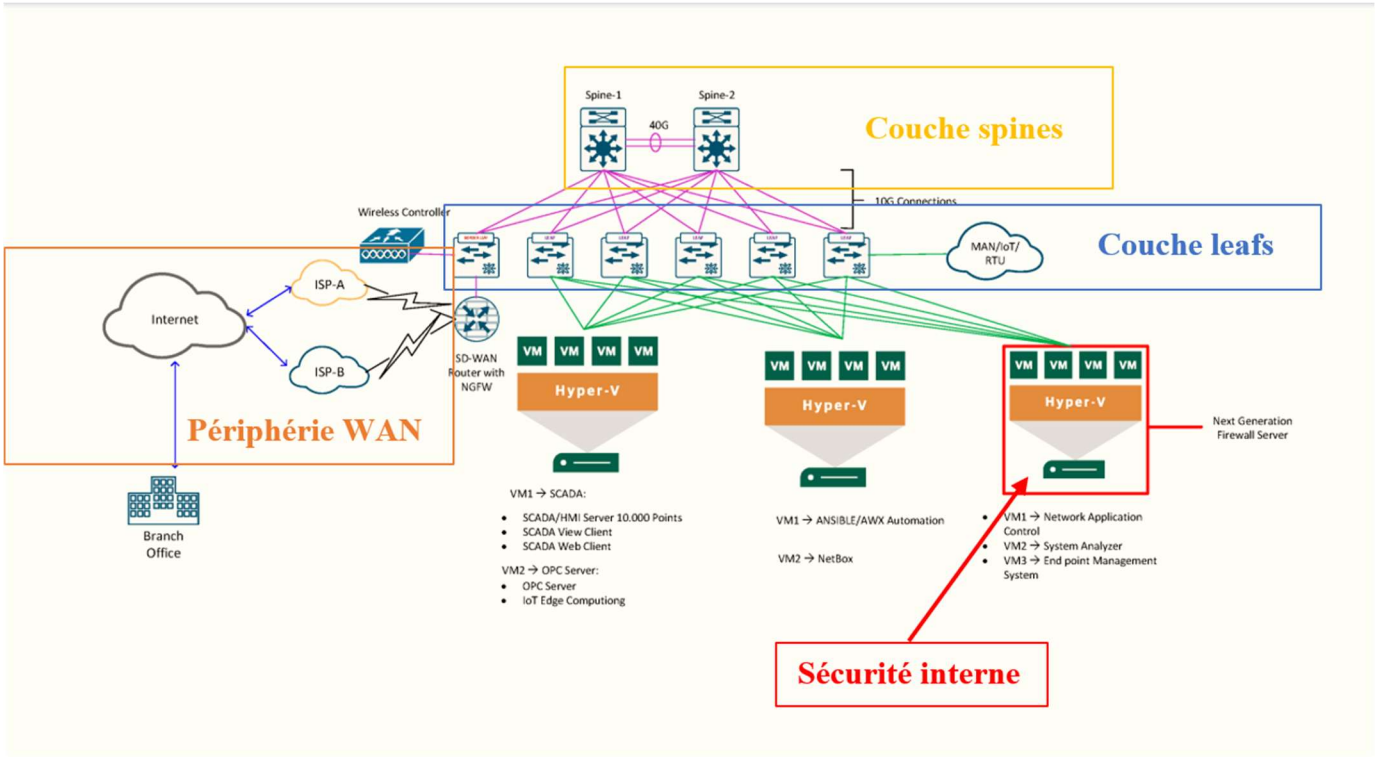


Figure 10 : Architecture de la topologie

La **couche Spine** est constituée de deux commutateurs Arista EOS interconnectés par un lien d'agrégation LACP. Ce lien inter-Spine assure la redondance et la propagation du plan de contrôle entre les deux équipements de la couche supérieure.

La **couche Leaf** compte six équipements Cisco IOS, chacun raccordé aux deux Spines par des liens point à point dédiés. Cette double connexion garantit qu'aucun équipement d'accès n'est un point de défaillance unique. L'un des Leafs joue le rôle de Border Leaf : il constitue le point d'entrée du trafic WAN dans la fabric et reste isolé des pare-feux internes, afin d'éviter toute boucle de routage sur le chemin WAN.

La **périphérie WAN** s'appuie sur un pare-feu FortiOS en mode SD-WAN, raccordé à deux opérateurs Internet distincts via quatre liens WAN, permettant à la fois la redondance et la distribution de charge. Ce dispositif est connecté au Border Leaf via un lien de données dédié.

La **sécurité interne** est assurée par une paire de pare-feux FortiOS en haute disponibilité, raccordée à chacun des équipements Leaf restants. Ces dispositifs assurent

l'inspection du trafic nord-sud des serveurs hébergés et constituent la passerelle par défaut des VLANs de workloads.

L'hétérogénéité des plateformes Cisco IOS, Arista EOS et FortiOS constitue une contrainte structurelle fondamentale. Ces trois familles présentent des syntaxes, des interfaces de gestion et des modèles de données distincts, ce qui rend impossible toute approche d'automatisation uniforme basée sur la seule ligne de commande.

4.2 Planification du trafic

L'infrastructure repose sur une séparation stricte entre trois plans de trafic aux rôles distincts : le plan de management, le plan de données et le plan de contrôle.

Le plan de management concentre l'ensemble des flux d'administration : accès SSH, appels API, exécution des playbooks Ansible et supervision. Il est maintenu sur un segment réseau dédié, physiquement et logiquement isolé du reste du trafic, afin de garantir l'accessibilité des équipements en toutes circonstances, y compris lors d'une perturbation du plan de données.

Le plan de données supporte le trafic applicatif, les liens point à point de la fabric et les segments de workloads. Il est structuré en sous-réseaux dédiés selon la nature des flux transportés.

Le plan de contrôle assure l'échange de routes entre équipements. Il repose sur OSPF, protocole retenu pour son support natif sur les trois plateformes et sa convergence rapide. Les identifiants de routage sont portés par des interfaces loopback, indépendantes de l'état des liens physiques.

Cette séparation constitue un prérequis architectural : elle garantit que les outils d'automatisation conservent en permanence un accès fiable aux équipements pour en auditer ou en corriger l'état, sans dépendre de la disponibilité du plan de données.

Le détail du plan d'adressage associé à chacun de ces plans est présenté au chapitre 3, dans le contexte de la mise en œuvre.

5. Choix des outils utilisés

Le choix des outils s'inscrit dans une logique de cohérence globale de la chaîne d'automatisation, tout en répondant aux exigences fonctionnelles et aux contraintes techniques identifiées précédemment. Chaque composant a été sélectionné pour son rôle spécifique dans l'architecture, sa compatibilité avec un environnement multi-constructeurs et sa capacité à s'intégrer dans une approche *Network as Code*.

Eve-NG : outil de simulation



Figure 11 : Logo eve-ng

Eve-NG s'est imposé ces dernières années comme une référence dans les environnements professionnels pour la simulation réseau. Il constitue l'environnement sur lequel repose la phase de test des configurations avant tout déploiement en production. Sa capacité à émuler des équipements multi-constructeurs (Cisco IOS/XE, Arista EOS, FortiOS) le rend cohérent avec l'architecture hétérogène du projet, permettant d'exécuter l'intégralité du pipeline dans un environnement isolé et maîtrisé. (EVE-NG LTD, 2023)

GNS3 a été écarté en raison d'une architecture moins stable sous forte charge et d'une intégration plus complexe avec les chaînes CI/CD.

NetBox : Référentiel de données d'infrastructure (SSoT)



Figure 12 : Logo de netbox

Le choix d'un référentiel centralisé étant fondamental dans une approche *Network as Code*, NetBox s'impose comme la solution la plus adaptée au contexte multi-constructeurs du projet. Son modèle de données natif couvre l'intégralité des ressources réseau (DCIM, IPAM, topologie), et ses interfaces REST et GraphQL permettent son interrogation directe par les composants aval du pipeline. (NetBox Labs, 2026)

Sa déployabilité via Docker et son système de plugins (notamment netbox-routing,s .f section suivante) garantissent une extensibilité maîtrisée sans dépendance à des composants propriétaires. Les CMDB développées sur mesure ont été écartées pour absence de standardisation, et des solutions comme NAPALM-Logs pour leur périmètre fonctionnel insuffisant.

Ansible : Moteur d'orchestration et de déploiement



Figure 13 : Logo d'Ansible

Ansible assume le rôle de moteur d'exécution des configurations sur les équipements cibles. Son architecture agentless élimine tout prérequis de déploiement sur les équipements, réduisant la complexité opérationnelle. Les modules officiels pour Cisco IOS, Arista EOS et FortiOS assurent une compatibilité native avec l'infrastructure, sans développement additionnel. (Red Hat, Inc., 2026)

Nornir, bien que plus performant en termes de parallélisation, implique une maîtrise approfondie de Python et une prise en main plus complexe. SaltStack, quant à lui, repose sur une architecture à agents incompatible avec les contraintes de déploiement du projet.

Batfish : Validation statique pré-déploiement



Figure 14 : Logo de Batfish

Batfish intervient comme couche de validation avant tout déploiement effectif. Il analyse les configurations générées pour détecter les anomalies de routage, les incohérences de filtrage et les erreurs de configuration sans nécessiter d'accès aux équipements de production. Cette capacité à simuler le comportement réseau *hors-ligne* constitue un garde-fou essentiel dans la chaîne CI/CD. (Fogel, Fung, & Pedrosa, 2015)

Forward Networks propose des fonctionnalités comparables, mais son caractère propriétaire est incompatible avec les exigences de maîtrise des coûts et d'ouverture du projet. Les scripts de validation ad hoc ne peuvent atteindre un niveau d'exhaustivité équivalent.

pyATS : Audit opérationnel pré et post-déploiement



Figure 15 : Logo de pyATS

pyATS, associé aux parseurs Genie, assure la phase d'audit opérationnel du pipeline. Il collecte l'état réel des équipements sous forme d'objets Python structurés, permettant d'effectuer des comparaisons automatisées entre l'état pré- et post-déploiement (*snapshot diff*). L'intégration d'Unicon étend cette capacité aux équipements non-Cisco, notamment Arista EOS.

NAPALM et Netmiko ont été écartés en raison de leurs capacités limitées en matière de structuration des données collectées et d'absence de mécanisme natif de comparaison d'états.

GitHub Actions : Orchestration du pipeline CI/CD

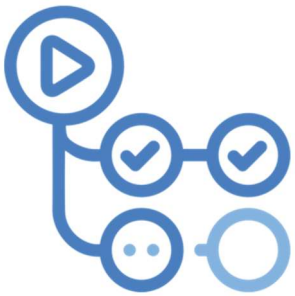


Figure 16 : Logo de GitHub Actions

GitHub Actions a été retenu pour son intégration native avec le dépôt Git du projet, permettant de définir l'ensemble des workflows d'automatisation directement en YAML, au sein du même référentiel que le code d'infrastructure. L'utilisation d'un *runner* auto-hébergé garantit la compatibilité avec l'environnement KVM local et évite l'exposition de données d'infrastructure à des runners publics. (GitHub, Inc., 2026)

Jenkins et GitLab CI, bien que matures, impliquent un effort de déploiement et de maintenance disproportionné au regard du périmètre du projet.

Git : Gestion des versions et mécanisme de rollback



Figure 17 : Logo de git

Git constitue le socle de traçabilité de l'ensemble du code d'infrastructure (playbooks, templates, inventaires). Il offre un historique complet et horodaté des modifications, et son intégration avec GitHub Actions permet le déclenchement automatique du pipeline à chaque

commit. En cas d'incident, la restauration d'un état antérieur via git revert constitue un mécanisme de rollback fiable et sans infrastructure additionnelle. (Cisco Systems, Inc., 2026)

Subversion (SVN) et les outils propriétaires de gestion de configuration ont été écartés pour leur moindre adoption dans les pratiques DevOps et leur intégration limitée avec les plateformes CI/CD modernes.

Docker : Conteneurisation des services d'infrastructure

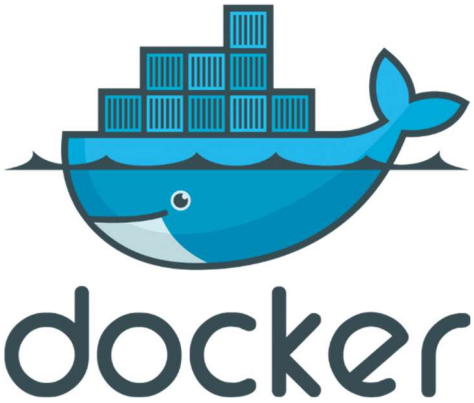


Figure 18 : Logo de docker

Docker assure le déploiement des services d'infrastructure du pipeline (NetBox, Batfish, Redis) dans des environnements reproductibles et isolés. L'usage d'images officielles et d'un fichier docker-compose standardisé garantit la portabilité de l'environnement et permet une reconstruction complète en cas de sinistre. (Docker, Inc., 2026)

LXC/LXD et les machines virtuelles complètes imposent une charge de maintenance supérieure et ne bénéficient pas de l'écosystème d'images publiques disponibles avec Docker.

6. Architecture de la solution

La solution est organisée autour d'un pipeline CI/CD structuré en trois phases principales (Pré-déploiement, Déploiement, Post-déploiement). Ces phases sont elles-mêmes constituées de cinq chaînes fonctionnelles qui s'enchaînent de manière cohérente pour transformer une intention métier en une configuration réseau opérationnelle, tout en garantissant la conformité avec l'état déclaré.

Chaîne de données

La chaîne de données constitue le point d'entrée unique de toute modification de l'infrastructure. Une source de vérité centrale (SSoT) stocke l'intégralité des informations nécessaires : équipements, interfaces, adresses IP, VLANs, routes statiques et paramètres de sécurité. Un mécanisme d'extraction automatique lit ces données et les structure en variables exploitables par les étapes suivantes. Ce couplage lâche entre la source de vérité et le processus de déploiement garantit que toute évolution du réseau est d'abord exprimée sous forme de données, avant d'être traduite en configuration.

Chaîne de génération

À partir des variables produites par la chaîne de données, la chaîne de génération élabore les configurations qui seront appliquées aux équipements. Pour les systèmes dont la configuration s'appuie sur des fichiers textuels, des modèles paramétrés par domaine fonctionnel interfaces, VLANs, protocole de routage, routage statique produisent les fichiers de configuration correspondants. Pour les systèmes pilotés par API, les paramètres de configuration sont directement transmis sous forme structurée sans génération de fichier intermédiaire. Cette approche duale permet de traiter de manière uniforme des équipements hétérogènes.

Chaîne de validation

La validation intervient à deux moments distincts du pipeline. En amont du déploiement, un validateur statique analyse les configurations produites : il modélise un réseau virtuel complet et vérifie l'absence d'erreurs structurelles, de boucles de routage ou d'incohérences. Ce contrôle est bloquant (quality gate) : tout défaut détecté interrompt le pipeline avant toute modification du réseau réel, constituant ainsi une barrière de prévention des incidents. En aval du déploiement, un mécanisme d'audit capture l'état opérationnel réel de chaque équipement et le compare à un état de référence capturé avant déploiement. Tout écart détecté est immédiatement signalé comme une dérive potentielle.

Chaîne de déploiement

La chaîne de déploiement applique les configurations validées sur les équipements cibles. Elle est conditionnée par la réussite de la validation statique et s'appuie sur les protocoles de communication appropriés à chaque famille d'équipements (connexion

sécurisée en ligne de commande pour les uns, interface de programmation REST pour les autres). L'enforcement de la conformité à la source de vérité est intégré à cette étape : les éléments absents de la source de vérité sont automatiquement supprimés des équipements, garantissant ainsi l'alignement permanent entre l'état déclaré et l'état réel.

Gestion du rollback et des incidents

La gestion des incidents repose sur deux mécanismes complémentaires :

- **Quality gate préventif** : la chaîne de validation constitue une barrière empêchant toute configuration défectueuse d'atteindre le réseau
- **Historique de versions** : le dépôt de code conserve un journal complet de tous les états successifs de l'infrastructure ; en cas d'incident post-déploiement, il suffit de réexécuter le pipeline à partir d'une version précédente pour restaurer un état antérieur connu

L'idempotence de l'ensemble de la chaîne garantit que ce retour arrière est aussi stable que le déploiement initial.

La figure suivante synthétise l'enchaînement au sein du pipeline CI/CD, de l'intention à la conformité :

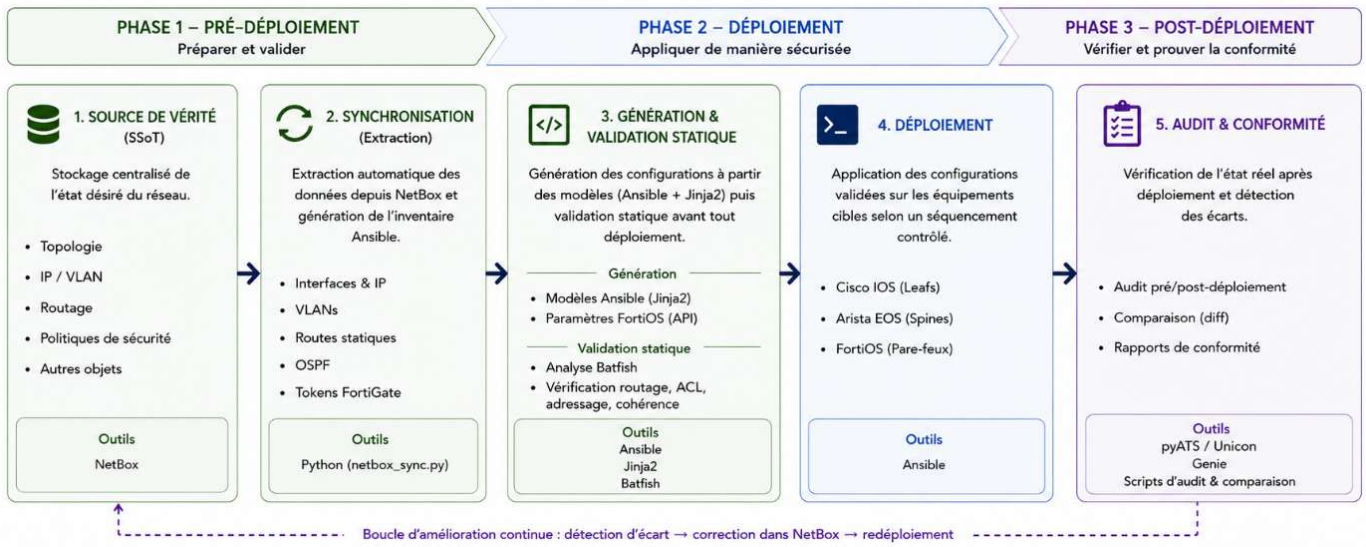


Figure 19 : Pipeline CI/CD d'automatisation réseau

7. Planification la mise en œuvre

La mise en œuvre de la solution suit une démarche agile structurée en cinq sprints de une à trois semaines chacun. Chaque sprint couvre un domaine fonctionnel distinct du pipeline et est défini par un livrable opérationnel concret, directement traçable aux exigences fonctionnelles spécifiées en section 2.3. Cette organisation assure une progression maîtrisée et un suivi rigoureux de chaque étape de la réalisation.

Le figure ci-dessous présente la planification retenue, avec pour chaque sprint l'incrément fonctionnel associé et le livrable attendu à l'issue de l'itération.

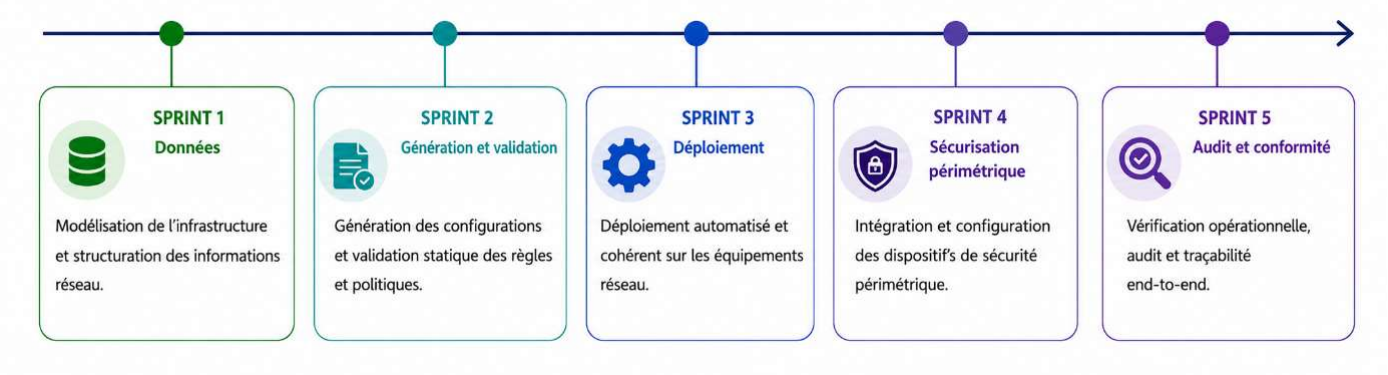


Figure 20 : Planification des cinq sprints

Le chapitre 3 suit cette organisation sprint par sprint, de la constitution de la source de vérité jusqu'à la validation de la conformité de bout en bout du pipeline.

8. Conclusion

Ce chapitre a permis de construire, à partir des concepts fondateurs du Network as Code et d'une analyse structurée des besoins, une architecture de solution complète et justifiée. L'organisation en chaînes fonctionnelles, la séparation des plans de trafic et les choix technologiques retenus en constituent un socle cohérent, modulaire et idempotent. Cette architecture, déclinée en une planification agile de cinq sprints, fournit le cadre directeur de la réalisation présentée au chapitre 3.

Chapitre 3 : Réalisation et tests

1. Introduction

Ce chapitre présente la mise en œuvre complète de la chaîne d'automatisation conçue au chapitre précèdent, de la source de vérité jusqu'à la validation post-déploiement. Organisé en cinq sprints, il adopte une démarche agile intégrée où chaque réalisation est immédiatement suivie de sa preuve de validation.

Ci-dessous chaque sprint et son backlog spécifique :

Tableau 5 : Présentation du backlog des cinq sprints

Sprint	Intitulé	Sections	Backlog
Sprint 1	Données	§2	Infrastructure de laboratoire et modélisation de la SSoT
Sprint 2	Génération et validation	§3	Synchronisation, génération configurations, validation Batfish
Sprint 3	Déploiement réseau	§4	Baseline pré-déploiement, déploiement Cisco IOS et Arista EOS
Sprint 4	Sécurisation périmétrique	§5	Déploiement FortiGate et idempotence
Sprint 5	Audit et conformité	§6	Audit post-déploiement et archivage

2. Sprint 1 : Modélisation des données et fondation de la source de vérité

Ce sprint établit les deux fondations du projet : l'environnement de laboratoire sur lequel s'exécute l'ensemble de la chaîne d'automatisation (§2.1 à §2.3), et la modélisation complète de l'infrastructure dans NetBox en tant que source de vérité unique (§2.4 à §2.6).

2.1 Serveur hôte et plans réseau

Le projet s'appuie sur une instance EVE-NG hébergée sur un serveur interne de l'entreprise, accessible à l'adresse 10.66.14.12, disposant d'environ **16 Go de RAM**, **250 Go de stockage** et **6 vCPU**. Les ressources disponibles ont été vérifiées directement depuis le shell du serveur :

```
root@eve-ng:~# free -h && df -h / && nproc
               total        used        free      shared  buff/cache   available
Mem:            15Gi        10Gi       1.4Gi       6.0Mi       3.9Gi       5.0Gi
Swap:          4.0Gi       564Mi       3.4Gi
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv  250G   45G  194G  19% /
6
root@eve-ng:~#
```

Figure 21 : Ressources du serveur EVE-NG

Le plan d'adressage de l'environnement matérialise la séparation stricte des trois plans de trafic définie architecturalement au chapitre 2 :

Tableau 6 : Plan d'adressage complet de l'environnement de laboratoire

Plan	**Plage d'adressage**	**Usage**
Accès externe	10.66.14.0/24	Accès SSH depuis le poste hôte vers un équipement de la topologie ; externe à la topologie EVE-NG
Management (OOB)	192.168.99.0/24 VLAN 99	SSH ou API REST depuis Ansible uniquement pour la supervision
Données (Data Plane)	10.0.0.0/8 (sous-réseaux dédiés)	Trafic applicatif, liens P2P fabric, VLANs serveur
Contrôle (Control Plane)	Loopbacks 10.1.0.0/24, OSPF Area 0	Échange de routes entre équipements, identifiants de routage
Liens WAN point à point	10.200.0.0/16	Interconnexion entre les routeurs ISP et le pare-feu SD-WAN

Le nœud Linux occupe une position charnière entre deux de ces plans. Son interface sur 10.66.14.13/24 le rattache au réseau physique de l'entreprise, permettant un accès SSH depuis le poste Windows indépendamment de l'état de la topologie.

Son interface sur 192.168.99.10/24 le positionne à l'intérieur du plan de management, depuis lequel transitent l'ensemble des flux d'automatisation. Ces deux interfaces servent des usages distincts et aucun flux du pipeline ne transite par l'interface d'accès externe.

2.2 Infrastructure Docker

L'infrastructure applicative est intégralement conteneurisée via Docker Compose sur le nœud Linux. Le stack regroupe **huit services actifs en permanence**, répartis en deux groupes fonctionnels.

Composants applicatifs NetBox (image personnalisée netbox-custom:latest) :

- **netbox** : interface Web et exposition des API REST et GraphQL
- **netbox-worker** : traitements asynchrones en arrière-plan via RQ (Redis Queue)
- **netbox-housekeeping** : tâches de maintenance planifiées

Services de support :

- **postgres** : base de données relationnelle
- **redis** : file de messages pour le worker
- **redis-cache** : couche de cache applicatif

L'image netbox-custom:latest dérive de l'image officielle NetBox et intègre le plugin **netbox-routing**, qui enrichit la plateforme d'une gestion des routes statiques exploitées dans le projet via GraphQL. L'ensemble des surcharges est isolé dans `docker-compose.override.yml`, laissant le fichier `docker-compose.yml` d'origine intact.
La persistance des données est assurée par six volumes Docker nommés :

- netbox-postgres
- netbox-redis-data et netbox-redis-cache-data
- netbox-media-files, netbox-reports-files et netbox-scripts-files

Des sondes *healthcheck* conditionnent le lancement de chaque service à la disponibilité effective de ses dépendances, garantissant notamment que netbox-worker ne démarre qu'une fois postgres et redis pleinement opérationnels

La figure suivante présente l'état et la santé des huit services au moment de la collecte :

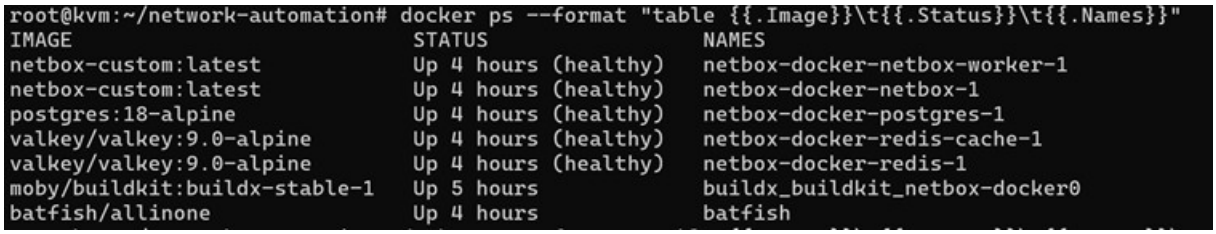


Figure 22 : Sortie de docker compose ps sur l'état et santé des huit services

Les fichiers `Dockerfile-Plugins` et `docker-compose.override.yml`, qui matérialisent la personnalisation de l'image NetBox et l'exposition du service, sont reproduits intégralement en Annexe A.1 (Figures 37 et 38).

2.3 Topologie EVE-NG et connectivité

La topologie déployée dans EVE-NG reproduit une architecture datacenter Spine/Leaf complétée d'un périmètre de sécurité FortiGate. L'ensemble partage le segment de management **192.168.99.0/24**.

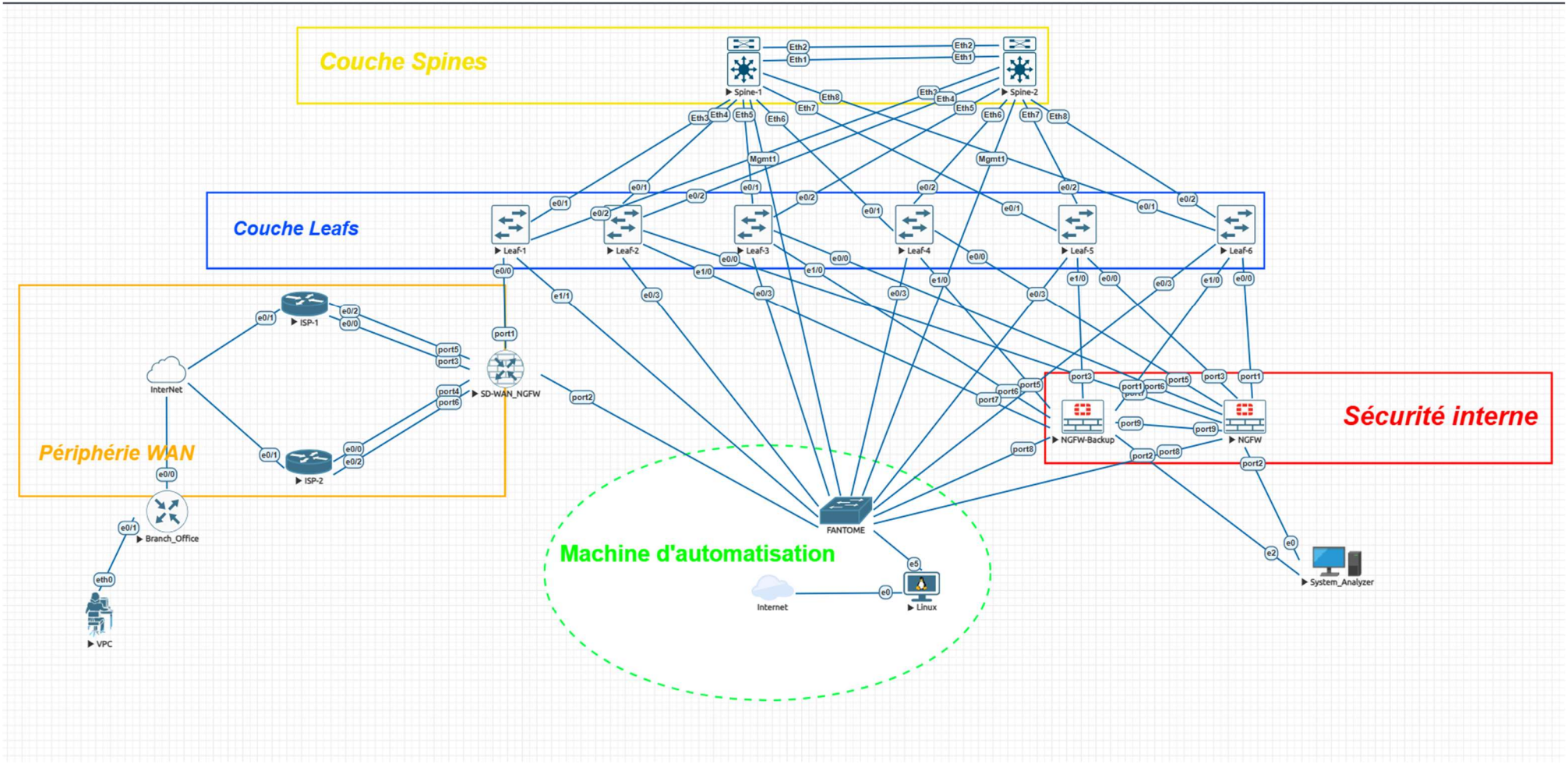


Figure 23 : Vue d'ensemble de la topologie EVE-NG

Tableau 7 : Inventaire des équipements simules dans EVE-NG

Équipement	**OS simulé**	**Périmètre Ansible**
Spine-1, Spine-2	Arista vEOS	Oui
Leaf-1 à Leaf-6	Cisco vIOS	Oui
SD-WAN_NGFW, NGFW, NGFW-Backup	FortiGate-VM	Oui
ISP-1, ISP-2	Cisco IOL	Non, configuration manuelle

Connectivité Internet

La sortie Internet transite par deux ISPs redondants, chacun relié au SD-WAN_NGFW par deux liaisons physiques, soit quatre liens WAN au total.

Tableau 8 : Details des liens entre les routeurs ISP et le pare-feu SD-WAN_NGFW

Lien	**Interface ISP**	**IP ISP**	**Interface SD-WAN**	**IP SD-WAN**	**Distance**
ISP-1 principal	Eth0/0	10.200.1.1/30	port3	10.200.1.2/30	10
ISP-1 redondant	Eth0/2	10.200.2.1/30	port5	10.200.2.2/30	20
ISP-2 principal	Eth0/0	10.200.3.1/30	port4	10.200.3.2/30	30
ISP-2 redondant	Eth0/2	10.200.4.1/30	port6	10.200.4.2/30	40

Les ISPs sont délibérément hors périmètre Ansible et jouent le rôle de FAI. Leur configuration est statique et fixe sur la durée du projet.

La configuration complète de ISP-1 (interfaces, ACL NAT et route par défaut) est documentée en Annexe A.2, Figure 39.

Plan de management

Le nœud Linux dispose de deux interfaces avec des rôles strictement séparés :

Tableau 9 : Interfaces du nœud d'automatisation Linux

Interface	**Réseau**	**Rôle**

eth0	10.66.14.13/24	OOB (Out of Band) avec accès SSH, et NetBox
eth5	192.168.99.10/24	Data plane : joignabilité des équipements simulés

Cette séparation est structurante pour le pipeline : Ansible atteint les équipements simulés exclusivement via eth5, tandis que NetBox est exposé sur eth0.

2.4 Modélisation des équipements

L'infrastructure est modélisée en trois niveaux hiérarchiques dans NetBox : sites, équipements et interfaces. L'adressage IP est géré via l'IPAM et le routage dynamique via le plugin netbox-routing. Aucune variable n'est définie localement dans Ansible ; l'ensemble des paramètres réseau est résolu à l'exécution par interrogation de l'API Netbox.

Au niveau équipement, le module DCIM décrit chaque nœud à travers un site unique, un rôle fonctionnel (spine, leaf, firewall, etc) et une plateforme explicite. Le champ Platform constitue le discriminant d'automatisation : il oriente le traitement vers la famille d'équipement concernée lors de la synchronisation et du déploiement

.

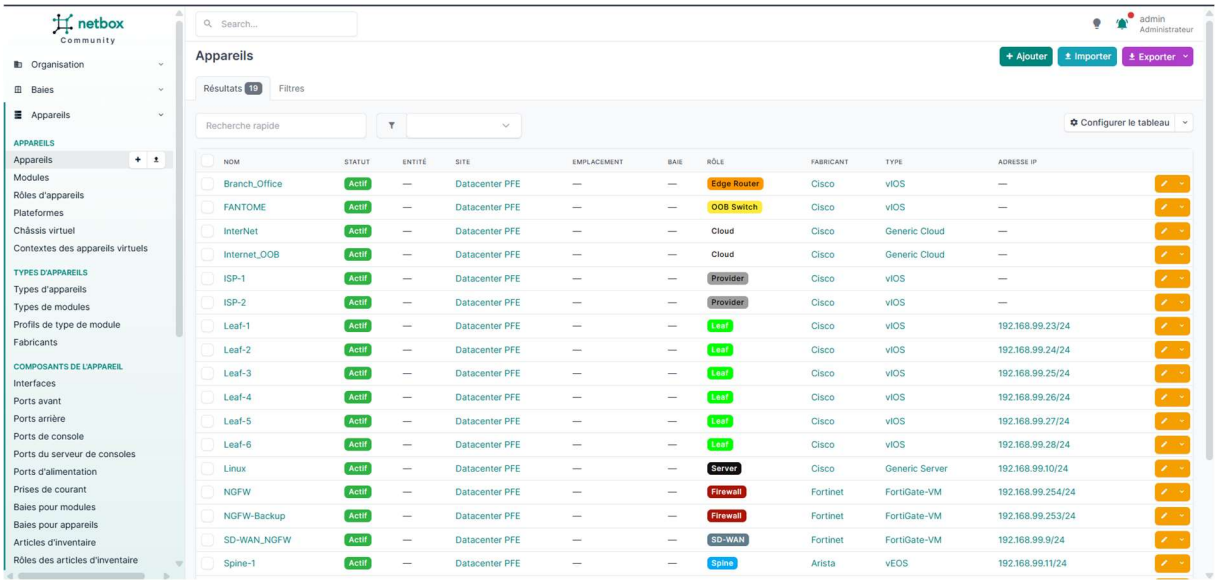


Figure 24 : Devices NetBox : rôles, plateformes et site

2.5 Modélisation des interfaces et de l'adressage

Le module IPAM de NetBox centralise les interfaces et leurs adresses IP. Chaque interface est typée (fabric, loopback, management, WAN) et rattachée à un sous-réseau dédié (192.168.99.0/24 pour le management et 10.0.0.0/8 pour le data plane, etc). Le champ primary_ip4 de chaque équipement est exploité par le script netbox_sync.py (cf. section 3.2.2) pour alimenter automatiquement l'inventaire Ansible en adresses de connexion.

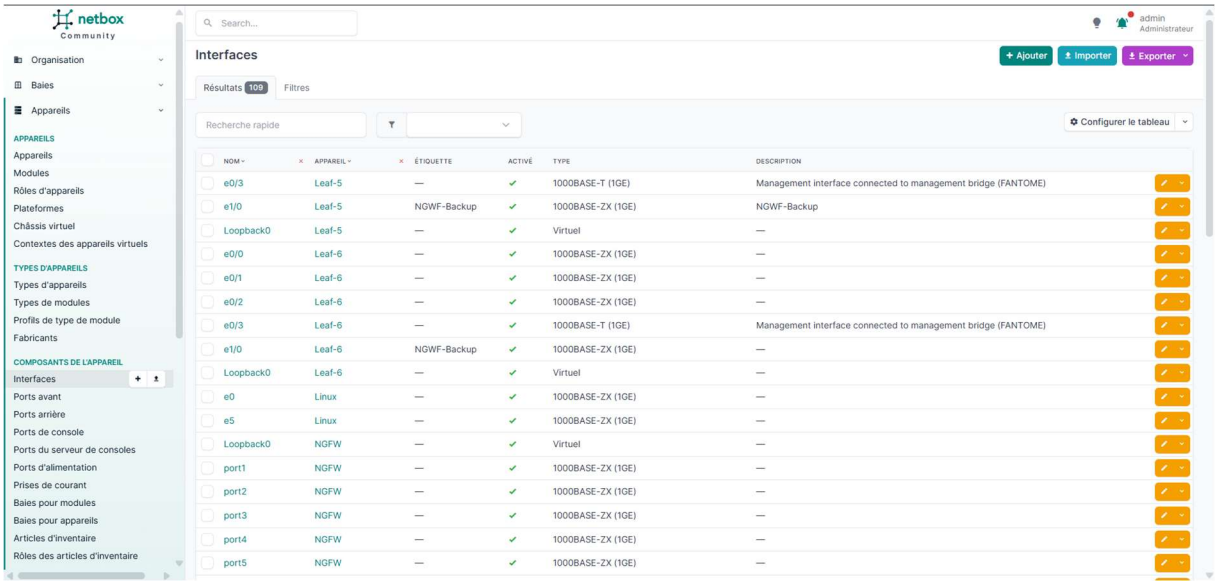


Figure 25 : Interfaces NetBox et adresses IP primaires

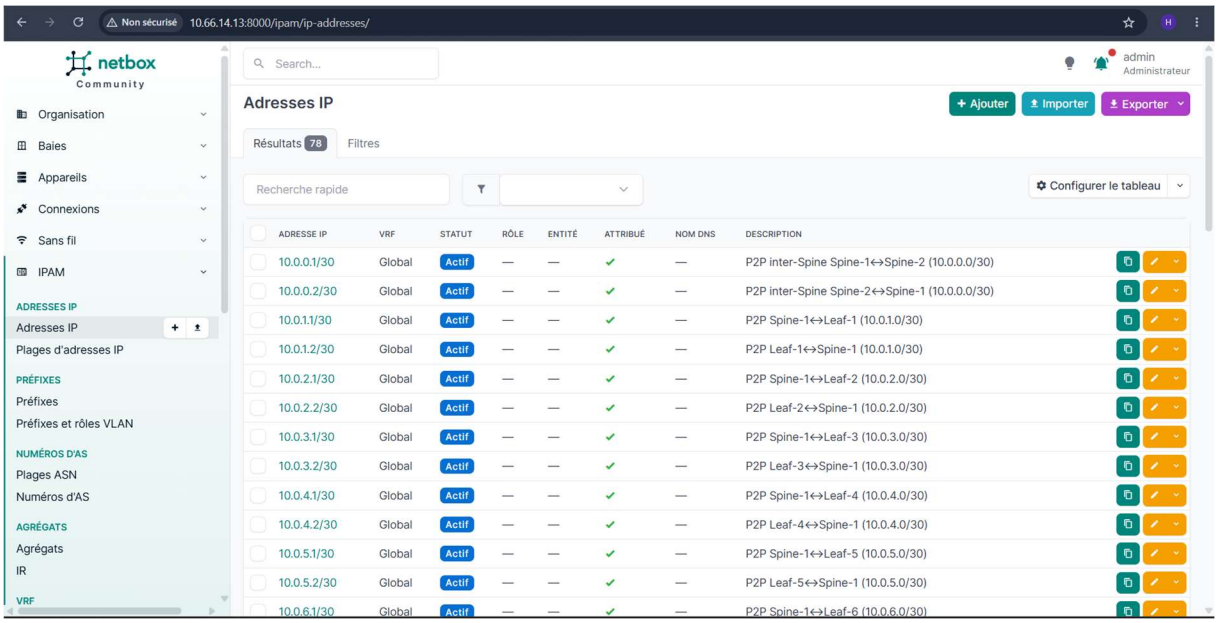


Figure 26 : Vue d'ensemble de la modélisation des Adresses IP

2.6 Modélisation du routage

Le plugin netbox-routing, intégré à l'image personnalisée netbox-custom:latest, étend NetBox avec la gestion des routes statiques. L'extraction de ces objets est réalisée via GraphQL, l'API REST du plugin ne restituant pas directement la ressource attendue.

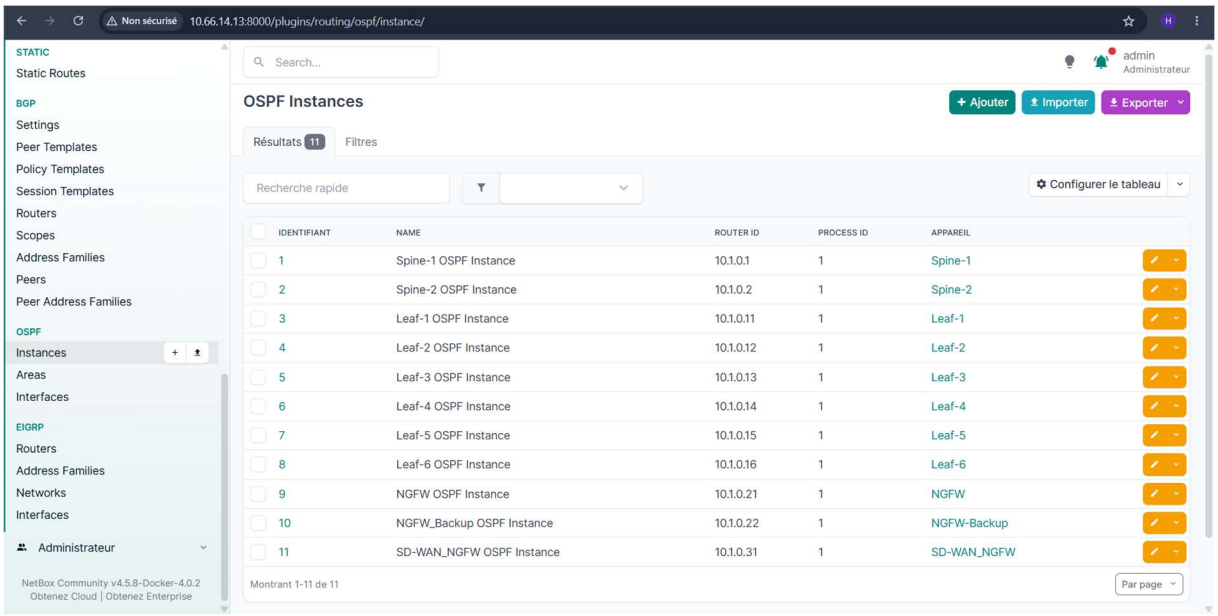


Figure 27 : Modélisation des instances ospf

Les routes statiques SD-WAN sont modélisées comme objets NetBox puis répercutées par `netbox_sync.py` dans le fichier `host_vars/SD-WAN_NGFW.yml`, avec leurs distances

administratives respectives. Ces variables sont consommées par le rôle `routing.yml` lors du déploiement. (cf. section 5.1) :

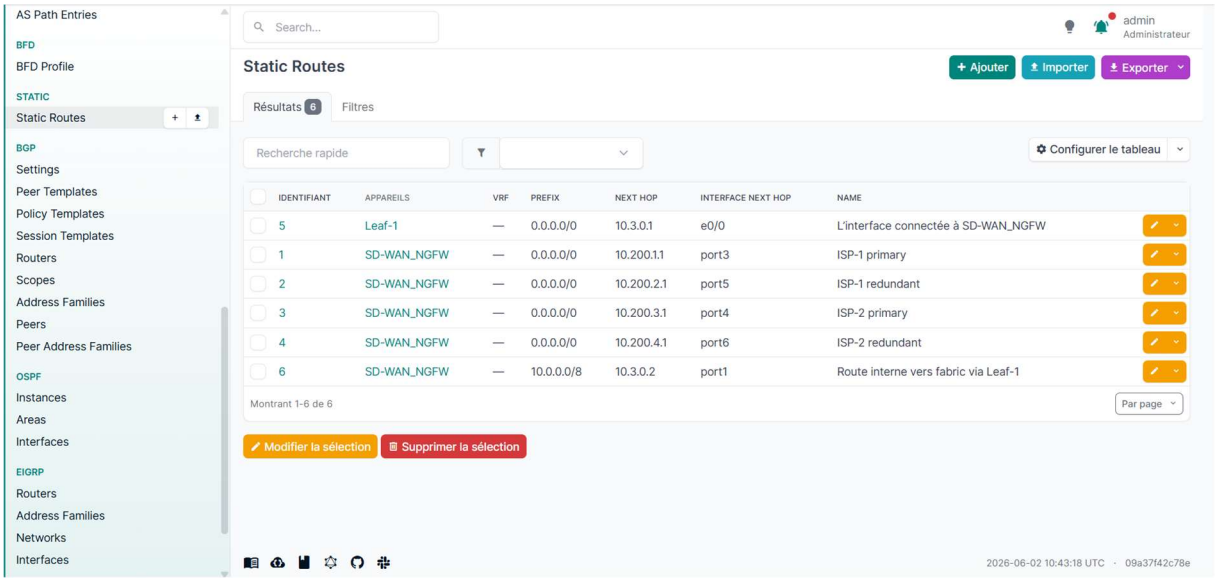


Figure 28 : Routes statiques SD-WAN déclarées dans NetBox

Validation du Sprint 1

L'environnement de laboratoire est pleinement opérationnel et la source de vérité NetBox est intégralement renseignée. Toutes les données d'infrastructure (équipements, interfaces, adressage, routes statiques SD-WAN et instances OSPF) sont accessibles via les API REST et GraphQL. La fondation de données du projet est posée.

3. Sprint 2 : Génération des configurations et validation statique

Ce sprint couvre l'ensemble de la chaîne de production et de validation des artefacts de configuration : de la définition de l'architecture d'automatisation (§3.1) à la génération des configurations (§3.3) en passant par la synchronisation de la SSoT (§3.2) et leur validation statique par Batfish (§3.4). Aucun équipement n'est encore contacté durant ce sprint.

3.1 Architecture du pipeline CI/CD

Le projet est versionné dans un dépôt Git unique structuré selon la séparation stricte des responsabilités. Chaque répertoire correspond à un rôle fonctionnel précis dans la chaîne d'automatisation.

Architecture globale du pipeline :

```
network-automation/
├── ansible.cfg  # Configuration Ansible (hosts, roles_path, vault)
├── group_vars/
│   └── all/
│       ├── netbox_token_vault.yml  # Token NetBox (chiffré AES-256)
│       └── fortigate_tokens_vault.yml  # Tokens API FortiGate
(chiffré AES-256)
│   ├── leafs.yml  # Paramètres connexion Cisco IOS
│   ├── spines.yml  # Paramètres connexion Arista EOS
│   └── fortigates.yml  # Paramètres connexion FortiOS via API REST
├── host_vars/  # ← GÉNÉRÉS par netbox_sync.py (ignorés Git)
│   ├── Leaf-1.yml ... Leaf-6.yml
│   ├── Spine-1.yml, Spine-2.yml
│   ├── NGFW.yml, NGFW-Backup.yml, SD-WAN_NGFW.yml
│   └── [19 équipements total]  — adresses, interfaces, routes,
tokens

├── inventory/                # Fichiers de topologie et inventaires
d'audit
│   ├── testbed_cisco.yaml   # Topologie pyATS pour équipements
Cisco IOS
│   ├── testbed_docker.yaml  # Topologie pyATS pour Arista EOS
(container)
│   └── testbed_arista.yaml  # Alternative de topologie Arista EOS

├── scripts/
│   ├── netbox_sync.py  # Synchronisation NetBox → host_vars +
inventaire hosts
│   ├── validate_batfish.py  #  Validation statique pré-déploiement
│   └── generate_pyats_testbed.py  #  Génération testbed pyATS
```

```
├── audits/
│   ├── arista_audit.py  # Audit Arista EOS post-déploiement
│   ├── fortigate_audit.py  # Audit FortiOS API REST
│   └── *_diff.py  # Comparaison baseline/post-deploy

├── playbooks/
│   ├── build_network.yml  # Phase PRE  : generation configs
(Jinja2)
│   ├── deploy_network.yml  # Phase DEPLOY : Cisco IOS + enforcement
VLANs
│   ├── deploy_spines.yml  # Phase DEPLOY : Arista EOS + LACP
│   └── deploy_fortigates.yml  # Phase DEPLOY : FortiOS API REST

├── roles/
│   ├── firewall_fortinet/  # Déploiement complet FortiOS (API REST)
│   │   └── tasks/
│   │       ├── system.yml, policies.yml, hardening.yml
│   │       └── interfaces.yml, routing.yml, dmz_vip.yml,
observability.yml
│   ├── ospf/
│   │   └── tasks/ + templates/  (cisco ospf.j2, arista_ospf.j2)
│   ├── vlans/  # Configuration VLANs (Cisco/Arista)
│   ├── interfaces/  # Interfaces L2/L3 + LACP
│   ├── routing/  # Routes statiques (Cisco/Arista)
│   └── security/  # ACLs

├── docs/
│   ├── architecture.md  # Référence technique complète
│   └── [autres guides techniques]

└── backups/  # Sauvegardes PostgreSQL NetBox
```

Trois points structurels sont à retenir.

Les **host_vars/** sont intégralement générés par `netbox_sync.py` à chaque exécution du pipeline. Ils constituent un artefact de synchronisation, non une source de

données, et sont explicitement exclus du versionnement Git. Toute modification manuelle dans ces fichiers serait écrasée au prochain déclenchement.

Les **secrets** sont versionnés sous forme chiffrée AES-256 dans `group_vars/all/` et ne sont jamais écrits en clair sur disque ni transmis en argument de commande. Leur déchiffrement est conditionné par la présence du fichier `.vault_pass` dans le workspace, supprimé inconditionnellement à l'issue de chaque run.

Le fichier **`ansible.cfg`** constitue le point d'ancrage de l'ensemble des exécutions Ansible du pipeline. Il déclare le chemin de l'inventaire généré, le répertoire des rôles `(roles/)` et le chemin du fichier de déchiffrement vault `(.vault_pass)`. Ce dernier paramètre est le plus critique : il conditionne mécaniquement le déchiffrement automatique des secrets AES-256 à chaque exécution Ansible, sans qu'aucun argument supplémentaire ne soit requis en ligne de commande.

Le pipeline garantit la présence de `.vault_pass` dans le workspace en amont de toute exécution Ansible, et l'étape POST-12 le supprime inconditionnellement à l'issue du run.

Le pipeline s'organise en trois phases séquentielles déclenchées par chaque événement push, par le webhook netbox ou manuellement sur le dépôt. Une phase ne s'exécute que si la précédente s'est terminée sans erreur.

Tableau 10 : Details des 10 etapes du pipeline CI/CD

Phase	Étape	Outil	Bloquant
PRE	PRE-1 : Synchronisation NetBox	`netbox_sync.py`	Oui
	PRE-2 : Génération des configurations	`build_network.yml`	Oui
	PRE-3 : Validation statique	`validate_batfish.py`	Oui
	PRE-4 : Génération testbed pyATS	`generate_pyats_testbed.py`	Oui
	PRE-5 : Audit baseline (Cisco / Arista / FortiOS)	pyATS / Unicon / API REST	Oui

DEPLOY	DEPLOY-6 : Déploiement Cisco IOS	`deploy_network.yml`	Oui
	DEPLOY-7 : Déploiement Arista EOS	`deploy_spines.yml`	Oui
	DEPLOY-8 : Déploiement FortiOS	`deploy_fortigates.yml`	Oui
POST	POST-9 : Audit post-déploiement	pyATS / Unicon / API REST	Oui
	POST-10 : Comparaison baseline / post-deploy	`*_diff.py`	Oui

Les lignes de détail internes figurent en Annexe B.1.

3.2 Synchronisation avec la source de vérité

Architecture fonctionnelle du script

`netbox_sync.py` constitue le point d'entrée de chaque exécution du pipeline. Avant toute extraction, il déchiffre à la volée deux vaults AES-256 versionnés dans `group_vars/all` : l'un fournissant le token NetBox, l'autre les tokens FortiGate indexés par équipement. Les secrets sont résolus en mémoire et ne transitent jamais en clair sur disque.
À partir des données extraites de NetBox, le script génère ensuite deux artefacts : un fichier `host_vars/<device>.yml` par équipement actif et le fichier d'inventaire `hosts`. Aucune donnée d'infrastructure n'est définie manuellement dans ces fichiers.

Extraction REST et GraphQL

L'API REST de NetBox est consommée via pynetbox. Pour chaque équipement dont le statut est actif, le script extrait :

- les interfaces (dcim.interfaces) avec leur mode VLAN, leur LAG et leur description ;
- les adresses IP (ipam.ip_addresses) associées à chaque interface ;
- les VLANs du site (ipam.vlans) injectés sous la clé global_vlans.

Les routes, extraites via GraphQL, sont filtrées par équipement et triées par distance administrative croissante avant injection dans les `host_vars/`. Les instances et interfaces OSPF sont associées à chaque équipement par jointure sur leurs identifiants.
Les routes WAN du SD-WAN_NGFW proviennent exclusivement aussi de NetBox.

Génération de l'inventaire Ansible

Les variables de connexion globales (ansible_user, ansible_connection, ansible_ssh_extra_args) sont écrites dans la section [all:vars] du fichier d'inventaire `hosts`.

Les équipements FortiGate font l'objet d'un traitement spécifique : leur interface de management est auto-détectée à partir de l'adresse IP primaire remontée par NetBox, et le token d'authentification API correspondant est extrait du vault puis injecté comme paramètre de connexion dédié à ce type de plateforme. Cette injection garantit que chaque appel REST ultérieur est authentifié sans intervention manuelle.

À l'issue du traitement de chaque device, le script génère le fichier hosts structuré en groupes fonctionnels : leafs, spines, fortigates, fortigates_ha_secondary et others.

Résultats de la synchronisation :

L'exécution du run #88 a produit l'inventaire suivant, archivé dans `/root/netdevops-archives/run-88/hosts` :

Commande exécutée :

```
cat /root/netdevops-archives/run-88/hosts
```

Sortie :

```
[leafs]

Leaf-1

Leaf-2

Leaf-3

Leaf-4

Leaf-5
```

```
Leaf-6



[spines]

Spine-1

Spine-2



[fortigates]

SD-WAN_NGFW

NGFW



[fortigates_ha_secondary]

NGFW-Backup

[all:vars]

ansible_user=admin

ansible_connection=network_cli

ansible_ssh_extra_args=-o StrictHostKeyChecking=no
```

Analyse :

L'inventaire reflète fidèlement la topologie modélisée dans NetBox. Les équipements sont classés en groupes fonctionnels exploitables par Ansible. La distinction entre fortigates (primaires) et fortigates_ha_secondary (secondaire) est assurée automatiquement par la détection des tags NetBox, garantissant que les playbooks de déploiement quotidien ne ciblent que les équipements primaires.

Contenu d'un fichier host_vars :

Chaque équipement actif dans NetBox possède un fichier `host_vars` dédié, généré intégralement par `netbox_sync.py`. L'exemple ci-dessous présente un extrait du fichier correspondant au FortiGate NGFW :

```
ansible_host: 192.168.99.254
```

```
netbox_role: firewall
netbox_platform: FortiOS
mgmt_interface: port8
ansible_httpapi_token: "{{ vault_fortigate_tokens['NGFW'] }}"
interfaces:
  Loopback0:
    ip_addresses: [10.1.0.21/32]
  port1:
    ip_addresses: [10.2.5.1/30]
    mode: management
  port3:
    ip_addresses: [10.200.1.2/30]
    description: ISP-1_principal
static_routes:
  - prefix: 0.0.0.0/0
    nexthop: 10.200.1.1
    distance: 10
  - prefix: 0.0.0.0/0
    nexthop: 10.200.2.1
    distance: 20
```

Analyse :

Chaque fichier `host_vars` constitue la projection directe des données NetBox pour un équipement donné. Ce mécanisme élimine tout risque d'incohérence entre la documentation et la configuration réellement déployée.

3.3 Génération des configurations

Le playbook **`build_network.yml`** consomme les `host_vars/` produits par `netbox_sync.py` et génère, via des templates Jinja2 organisés par famille d'équipements, les fichiers de configuration destinés à la validation statique puis au déploiement.

Périmètre d'exécution :

- Le playbook s'exécute intégralement sur le nœud d'automatisation (connection: local), sans aucune connexion aux équipements réels
- Les FortiGates sont exclus de cette phase : leur configuration est appliquée directement via API REST en phase DEPLOY

- Le rôle `ospf` est également absent : la configuration OSPF est injectée dynamiquement lors du déploiement par des modules natifs Ansible

Gestion de l'hétérogénéité Cisco IOS / Arista EOS :

- Les templates Jinja2 ne contiennent aucune logique de branchement entre les deux OS
- Un bloc de normalisation unique convertit les noms d'interfaces hétérogènes issus de NetBox vers une forme canonique avant génération des commandes
- L'interprétation finale est assurée par le module approprié (ios_config ou eos_config) sur la base de la variable ansible_network_os

Cette approche évite toute duplication de code entre familles d'équipements et permet l'intégration d'un nouvel OS sans modification des templates.

L'étape PRE-2 exécute le playbook `build_network.yml`. Elle consomme les `host_vars/` produits par `netbox_sync.py` et génère, via des templates Jinja2, les fichiers de configuration destinés à la validation statique puis au déploiement.

Artefacts de build produits :

L'exécution du run #88 a généré les fichiers de configuration suivants :

Commande exécutée :

```
ls /root/netdevops-archives/run-88/generated_configs/
```

Sortie (extrait) :

```
Leaf-1_interfaces.cfg    Leaf-4_interfaces.cfg    Spine-
1_interfaces.cfg

Leaf-1_vlans.cfg         Leaf-4_vlans.cfg         Spine-
1_vlans.cfg

Leaf-1_static_routes.cfg Leaf-5_interfaces.cfg    Spine-
2_interfaces.cfg

Leaf-2_interfaces.cfg    Leaf-5_vlans.cfg         Spine-
2_vlans.cfg

Leaf-2_vlans.cfg         Leaf-6_interfaces.cfg
```

```
Leaf-3_interfaces.cfg    Leaf-6_vlans.cfg
```

Analyse :

Le playbook `build_network.yml` produit un ensemble de fichiers de configuration par équipement. Chaque fichier correspond à une section fonctionnelle distincte (interfaces, VLANs, routes statiques), ce qui facilite le versionnement et le diagnostic ciblé. La présence de ces artefacts avant tout déploiement garantit que l'intention exprimée dans NetBox est correctement traduite en syntaxes exploitables par les équipements cibles.

3.4 Validation statique

Le script **`validate_batfish.py`** constitue un verrou bloquant de la phase PRE avant l'audit pré-déploiement. Il soumet les configurations générées au moteur Batfish, qui modélise le plan de contrôle sans interagir avec les équipements réels. Son résultat est binaire et bloquant : un fichier sentinelle `.validation_passed` est créé en cas de succès et conditionne mécaniquement le déclenchement de la phase DEPLOY.

Le script exécute deux contrôles séquentiels sur les configurations générées :

- ✓ vérification du statut de parsing (fileParseStatus) : tout fichier en état d'échec est identifié immédiatement ;
- ✓ identification des avertissements et erreurs de syntaxe (initIssues) : un mécanisme de filtrage distingue les faux positifs structurels d'EVE-NG des anomalies réellement bloquantes ;

L'environnement EVE-NG introduit structurellement des avertissements sans représentativité sur le réseau réel : équipements bridge (FANTOME, InterNet, Internet_OOB) et fichiers de configuration fragmentés sans directive hostname. Ces faux positifs sont connus et neutralisés par des masques Pandas appliqués sur le DataFrame initIssues dont les noms sont ci-dessous :

```
filtered = init_issues[
    ~(mask_internet | mask_no_hostname |
mask_invalid_interface_virtual)
]
```

La règle est stricte : une erreur d'interface sur une Leaf ou un Spine n'est jamais filtrée, car ces masques ne s'appliquent qu'aux fichiers des équipements bridge virtuels. Toute anomalie résiduelle dans *filtered* bloque immédiatement le pipeline.

Le fichier sentinelle est alors géré de manière conditionnelle comme le montre le script extrait de `validate_batfish.py` ci-dessous :

```
sentinel_path = os.path.join(snapshot_dir, ".validation_passed")

if validation_passed:
    with open(sentinel_path, 'w') as f:
        f.write("ok")
    logger.info("Validation Batfish réussie - fichier sentinel
créé.")
else:
    if os.path.exists(sentinel_path):
        os.remove(sentinel_path)
    logger.error("Validation Batfish échouée - déploiement bloqué.")
```

Résultat de la validation :

Pour ce #run-88, l'ensemble des fichiers de configuration générés pour les équipements du périmètre Ansible a été parsé avec succès. Le fichier sentinelle `.validation_passed` a été créé, autorisant le passage à la phase DEPLOY.

Validation du sprint 2

Les configurations réseau sont générées depuis la SSoT et validées statiquement par Batfish. Aucune anomalie bloquante n'a été détectée. Le fichier sentinelle `.validation_passed` *autorise le déclenchement du déploiement. Les artefacts de build et le rapport de validation sont archivés pour traçabilité.*

4. Sprint 3 : Déploiement réseau et validation opérationnelle

Ce sprint exécute le déploiement sur les équipements réseau Cisco IOS et Arista EOS. Il commence par la collecte de la baseline opérationnelle pré-déploiement (§4.1), qui constitue la référence de conformité pour la phase POST. Le déploiement est ensuite conditionné à la présence du fichier sentinelle Batfish (§3.4) avant d'être exécuté (§4.3).

4.1 Collecte de la baseline

Avant tout déploiement, la phase PRE-5 collecte l'état opérationnel de l'infrastructure ; cette baseline constitue l'unique référence pour la mesure de conformité en phase POST. Les trois familles d'équipements imposent des vecteurs de collecte distincts en raison de contraintes de compatibilité propres à chaque plateforme émulée dans EVE-NG.

Cisco IOS

La variable netbox_platform, injectée par `netbox_sync.py`, détermine le type pyATS assigné à chaque équipement dans `inventory/*.yml`. La collecte s'effectue via pyats learn, qui connecte chaque Leaf via Unicon et parse les sorties avec les parsers Genie natifs IOS, produisant des structures de données normalisées et sérialisées dans `snapshots/baseline/`. Dans le workflow du pipeline, la commande exécutée dans cette étape est celle-ci-dessous :

```
pyats learn platform interface \
  --testbed-file inventory/testbed_cisco.yaml \
  --output snapshots/baseline
```

pyats learn produit des structures de données normalisées indépendantes de la version IOS, sérialisées dans `snapshots/baseline/`.

Arista EOS

Genie ne dispose pas d'implémentation pour EOS dans cette version. La collecte Arista repose donc sur Unicon pour la connexion et sur du parsing par expression régulière pour l'exploitation des sorties. Par ailleurs, les dépendances SSH nécessaires à vEOS ne sont disponibles que dans le container officiel ciscotestautomation/pyats:latest-full, d'où

l'exécution containerisée. Dans le workflow du pipeline, la commande exécutée dans cette étape est celle-ci-dessous :

```
docker run --rm --network host \
  -v inventory/testbed_docker.yaml:/testbed.yaml:ro \
  -v scripts/audits/arista_audit.py:/arista_audit.py:ro \
  -v snapshots/arista_baseline:/output \
  ciscotestautomation/pyats:latest-full \
  bash -c "python3 /arista_audit.py /output"
```

Un fichier testbed statique dédié (inventory/testbed_docker.yaml) spécifie les options de négociation SSH requises par vEOS.
Ces paramètres ne peuvent pas être générés dynamiquement par `generate_pyats_testbed.py` et justifient le maintien d'un fichier dédié.

À l'intérieur du container, `arista_audit.py` établit la connexion via Unicon et collecte les états par device.execute(), Genie n'étant pas applicable :

FortiOS

Le script `fortigate_audit.py` opère exclusivement via API REST. Les FortiGates sont auto-découverts par lecture des `host_vars/` : tout hôte portant la variable ansible_httpapi_token (injecté par `netbox_sync.py`) est identifié comme FortiGate. Huit ressources sont collectées par équipement, correspondant exactement aux objets déployés par le rôle `firewall_fortinet` du run 88.

Résultats de la baseline :

Cisco IOS (Leafs)

La collecte s'effectue via pyats learn, qui connecte chaque Leaf via Unicon et parse les sorties avec les parsers Genie natifs IOS.

Commande éxecutée :

```
python3 -c "
import json
with open('/root/netdevops-archives/run-88/baseline/cisco_baseline.json') as
f:
    data = json.load(f)
```

```
for dev in ['Leaf-1', 'Leaf-2']:
    ospf = data[dev].get('ospf', {})
    print(dev, ':', json.dumps(ospf.get('neighbors', {}), indent=2))
"
```

Sortie (extrait Leaf-1) :


```
{
  "10.1.0.201": {
    "state": "FULL/DR",
    "address": "10.0.1.1",
    "interface": "GigabitEthernet0/1",
    "dead_time": "00:00:35"
  },
  "10.1.0.202": {
    "state": "FULL/DR",
    "address": "10.0.2.1",
    "interface": "GigabitEthernet0/2",
    "dead_time": "00:00:38"
  }
}
```


Analyse :

Chaque Leaf dispose, au moment de la collecte de la baseline, d'adjacences OSPF établies en état FULL avec les deux Spines (10.1.0.201 et 10.1.0.202). Cette convergence préalable confirme que l'environnement de départ est fonctionnel et que le déploiement pourra être mesuré par rapport à un état réseau stable.

Arista EOS (Spines)

Les six Leafs (Leaf-1 à Leaf-6) apparaissent comme voisins OSPF en état FULL sur Spine-1, les adjacences étant établies via les interfaces Ethernet3 à Ethernet8 :

Commande exécutée :

```
python3 -c "
import json
```

```
with open('/root/netdevops-archives/run-
88/arista_baseline/arista_audit.json') as f:
    data = json.load(f)
for dev in ['Spine-1', 'Spine-2']:
    print(dev, json.dumps(data[dev].get('ospf_neighbors', ''),
indent=2))
"
```

Sortie (extrait Spine-1) :


```
{
  "10.1.0.11": { "state": "FULL/BDR", "address": "10.0.1.2", "interface": "Ether
net3" },
  "10.1.0.12": { "state": "FULL/BDR", "address": "10.0.2.2", "interface":
"Ethernet4" },
  "10.1.0.13": { "state": "FULL/BDR", "address": "10.0.3.2", "interface":
"Ethernet5" },
  "10.1.0.14": { "state": "FULL/BDR", "address": "10.0.4.2", "interface":
"Ethernet6" },
  "10.1.0.15": { "state": "FULL/BDR", "address": "10.0.5.2", "interface":
"Ethernet7" },
  "10.1.0.16": { "state": "FULL/BDR", "address": "10.0.6.2", "interface":
"Ethernet8" }
}
```


La cohérence des adresses de loopback (10.1.0.11 à 10.1.0.16) avec la modélisation NetBox confirme l'intégrité de la synchronisation SSoT.

FortiOS (NGFW)

La collecte FortiOS s'effectue via API REST par `fortigate_audit.py`. Huit ressources sont collectées par équipement, correspondant exactement aux objets déployés par le rôle `firewall_fortinet`.

Commande exécutée :

```
python3 -c "
import json
with open('/root/netdevops-archives/run-
88/fortigate_baseline/fortigate_audit.json') as f:
```

```
    data = json.load(f)
for p in data['NGFW']['policies']:
    print(p['policyid'], p['name'], p['srcaddr'], p['dstaddr'],
p['service'])
"
```

Sortie :

```
5    Allow-Leafs-to-NGFW-Policy5  ['Leafs-Subnets']  ['Loopback0_NGFW']
['PING','SSH','SNMP']

10   Allow_Internal_Web           ['all']             ['all']
['HTTP','HTTPS']

20   Allow_DMZ_DNS                ['all']             ['all']
['DNS']

9999 BASELINE_DENY_ALL            ['all']             ['all']
['ALL']
```

Analyse :

L'ordre des politiques (5, 10, 20, 9999) est conforme à la stratégie définie dans le rôle `firewall_fortinet`. La politique 5 restreint l'accès au groupe d'adresses Leafs-Subnets, calculé automatiquement depuis NetBox. La politique BASELINE_DENY_ALL en dernière position garantit le principe de moindre privilège.

4.2 Séquencement et conditions d'exécution

La phase DEPLOY enchaîne six étapes ordonnées. Pour Cisco IOS et Arista EOS, chaque playbook conditionne son exécution à la présence du fichier sentinelle `.validation_passed` produit en PRE-3 par Batfish ; son absence interrompt immédiatement le pipeline.

Tableau 11 : Sequencement des six etapes de la phase DEPLOY

Étape	Playbook	Cible
6a	`deploy_network.yml`	Leafs Cisco IOS
6b	`deploy_routes.yml`	Routes statiques Cisco IOS
7a	`deploy_spines.yml`	Spines Arista EOS
7b	`deploy_ospf.yml`	OSPF multi-plateforme

8a	`deploy_fortigates.yml`	FortiGates via API REST
8b	sleep 30	Attente convergence LACP/STP

Le mécanisme de gate est implémenté dans chaque playbook de déploiement, comme le montre l'extrait du fichier *`validate_batfish.py`* ci-dessous :

```
- name: "GATE | Vérifier la présence du sentinel Batfish"
  stat:
    path: "{{ validation_marker }}"
  delegate_to: localhost
  register: validation_status

- name: "GATE | Bloquer si validation Batfish absente ou échouée"
  fail:
    msg: "Sentinel Batfish introuvable : {{ validation_marker }}"
  when: not validation_status.stat.exists
```

4.3 Déploiement Cisco IOS et Arista EOS

Le déploiement repose sur une architecture à rôles, chaque rôle couvrant un périmètre fonctionnel distinct de la configuration réseau. Cette organisation garantit la séparation des responsabilités et facilite l'extension du pipeline à de nouveaux équipements ou protocoles sans modification des playbooks existants.

`deploy_network.yml` et **`deploy_spines.yml`** poussent respectivement les configurations textuelles générées en PRE-2 sur les Leafs et les Spine. **`deploy_ospf.yml`** applique ensuite le rôle `ospf` sur l'ensemble des équipements, les données provenant des `host_vars/` alimentés depuis NetBox ; le rôle dispatche le traitement selon la variable netbox_platform, les trois familles exposant des syntaxes de configuration distinctes.
Pour Cisco IOS et Arista EOS, chaque playbook de déploiement conditionne son exécution à la présence du fichier sentinelle `.validation_passed` produit en PRE-3 par Batfish.

À titre d'illustration, le déploiement des routes statiques sur les Leafs Cisco charge la variable static_routes depuis les `host_vars/` et applique les entrées via ios_static_routes.

Les préfixes CIDR issus de NetBox sont convertis en notation réseau/masque par les filtres ipaddr.

```
- name: "Appliquer les routes statiques Cisco IOS"
  cisco.ios.ios_static_routes:
    config:
      - address_families:
          - afi: ipv4
            routes:
              - dest: "{{ item.prefix | ipaddr('network') }}"
                mask: "{{ item.prefix | ipaddr('netmask') }}"
                next_hops:
                  - forward_router_address: "{{ item.nexthop }}"
                    admin_distance: "{{ item.distance |
default(1) }}"
  loop: "{{ static_routes }}"
  when: static_routes is defined
```

Résultat du déploiement :

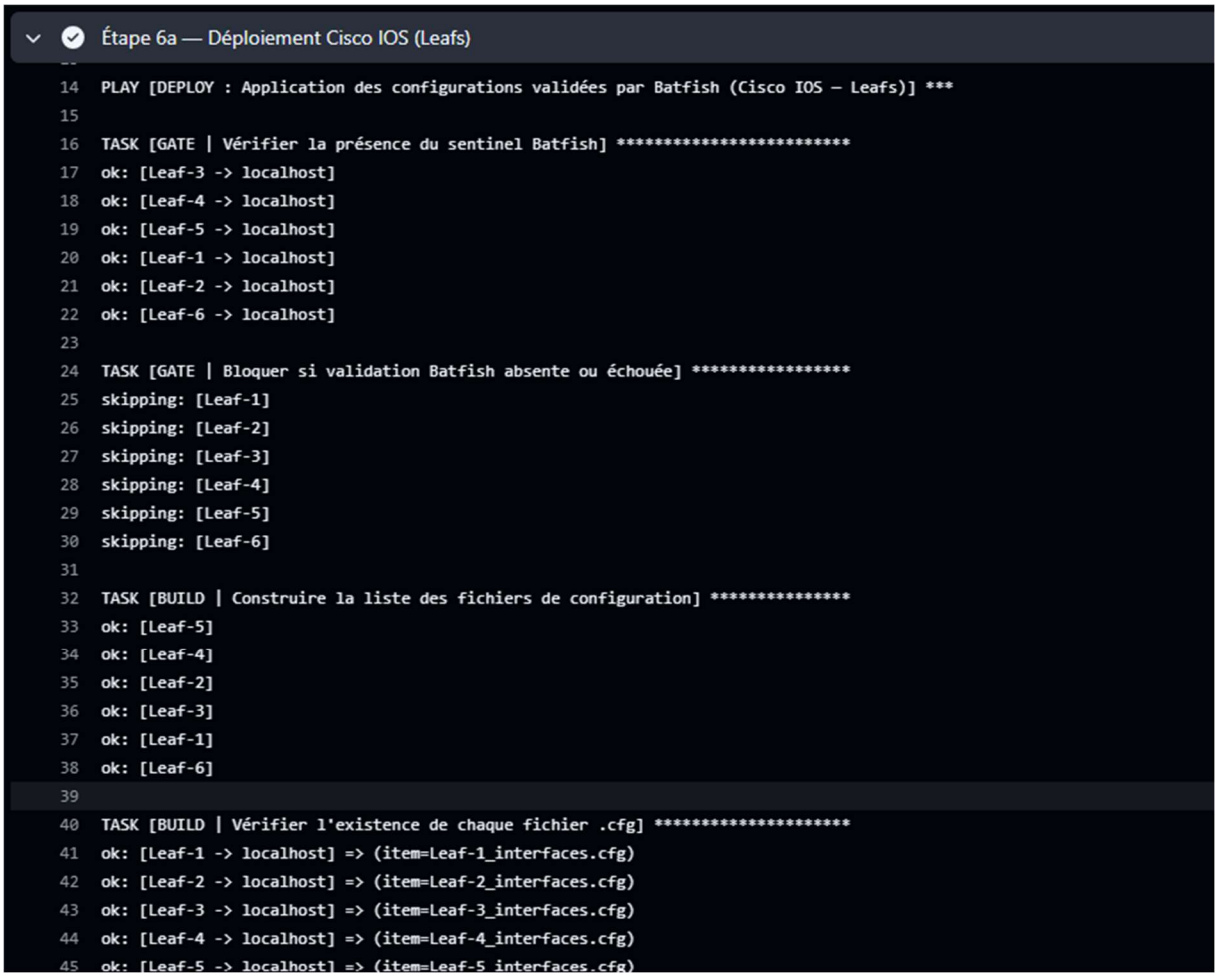
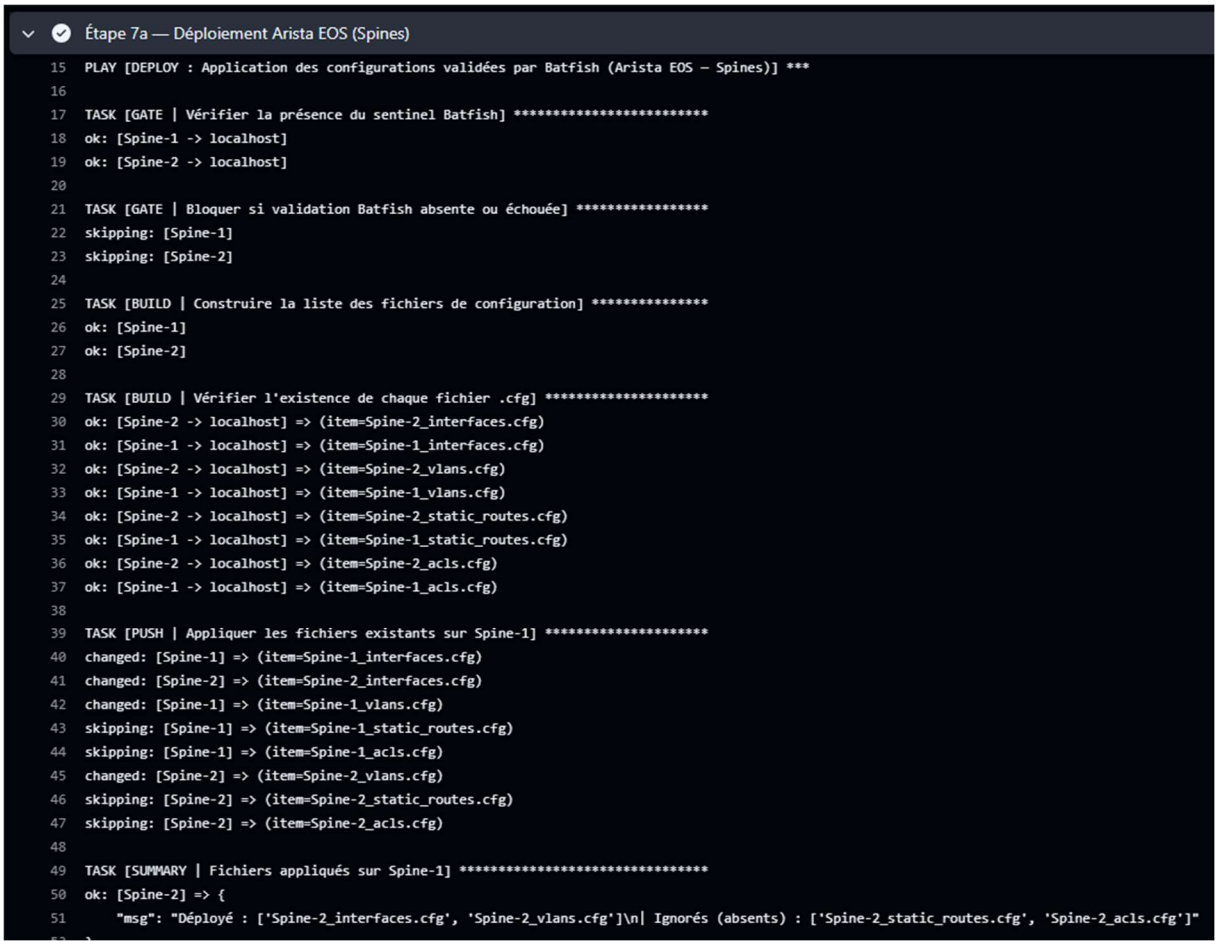


Figure 29 : Extrait de la sortie de l'etape 6a du pipeline



```
Étape 7a — Déploiement Arista EOS (Spines)
PLAY [DEPLOY : Application des configurations validées par Batfish (Arista EOS – Spines)] ***

TASK [GATE | Vérifier la présence du sentinel Batfish] ************************
ok: [Spine-1 -> localhost]
ok: [Spine-2 -> localhost]

TASK [GATE | Bloquer si validation Batfish absente ou échouée] *****************
skipping: [Spine-1]
skipping: [Spine-2]

TASK [BUILD | Construire la liste des fichiers de configuration] ***************
ok: [Spine-1]
ok: [Spine-2]

TASK [BUILD | Vérifier l'existence de chaque fichier .cfg] *********************
ok: [Spine-2 -> localhost] => (item=Spine-2_interfaces.cfg)
ok: [Spine-1 -> localhost] => (item=Spine-1_interfaces.cfg)
ok: [Spine-2 -> localhost] => (item=Spine-2_vlans.cfg)
ok: [Spine-1 -> localhost] => (item=Spine-1_vlans.cfg)
ok: [Spine-2 -> localhost] => (item=Spine-2_static_routes.cfg)
ok: [Spine-1 -> localhost] => (item=Spine-1_static_routes.cfg)
ok: [Spine-2 -> localhost] => (item=Spine-2_acls.cfg)
ok: [Spine-1 -> localhost] => (item=Spine-1_acls.cfg)

TASK [PUSH | Appliquer les fichiers existants sur Spine-1] *********************
changed: [Spine-1] => (item=Spine-1_interfaces.cfg)
changed: [Spine-2] => (item=Spine-2_interfaces.cfg)
changed: [Spine-1] => (item=Spine-1_vlans.cfg)
skipping: [Spine-1] => (item=Spine-1_static_routes.cfg)
skipping: [Spine-1] => (item=Spine-1_acls.cfg)
changed: [Spine-2] => (item=Spine-2_vlans.cfg)
skipping: [Spine-2] => (item=Spine-2_static_routes.cfg)
skipping: [Spine-2] => (item=Spine-2_acls.cfg)

TASK [SUMMARY | Fichiers appliqués sur Spine-1] ******************************
ok: [Spine-2] => {
    "msg": "Déployé : ['Spine-2_interfaces.cfg', 'Spine-2_vlans.cfg']\n| Ignorés (absents) : ['Spine-2_static_routes.cfg', 'Spine-2_acls.cfg']"
```

Figure 30 : Extrait de la sortie de l'etape 7a du pipeline

Analyse :

L'ensemble des tâches de déploiement s'est exécuté avec succès sur les six Leafs Cisco IOS et les deux Spines Arista EOS. Le compteur changed=0 sur chaque équipement dans certaines étapes du playbook confirme que l'état en place était déjà conforme à l'état désiré, ce qui démontre l'idempotence des playbooks lors d'exécutions successives. Aucune interruption ni erreur de connexion n'a été enregistrée sur les huit équipements ciblés.

4.4 Conformité post-déploiement Cisco et Arista

Cisco IOS

L'étape POST-10a exécute la comparaison native pyATS entre les snapshots baseline et post-déploiement des six Leafs. Cette étape est délibérément non bloquante : un écart éventuel est consigné dans les logs sans interrompre le pipeline.

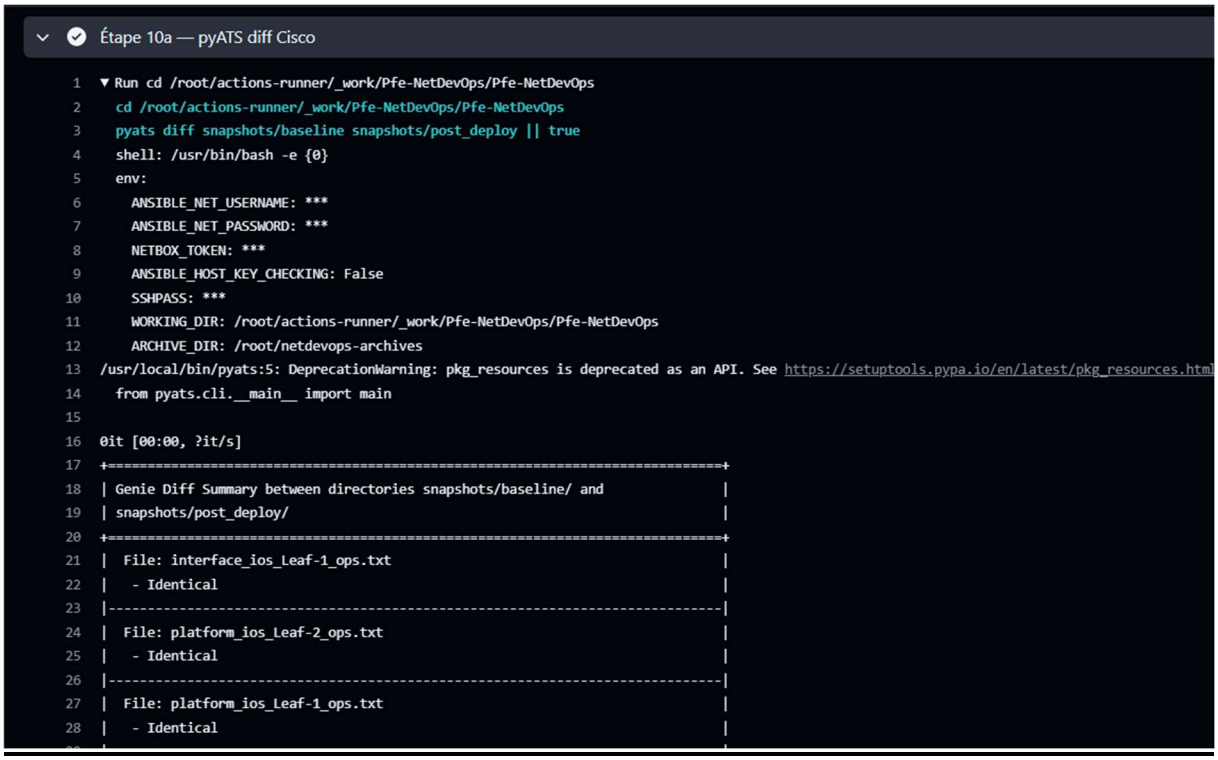


Figure 31 : Extrait de l'exécution de l'étape 10a sur le pipeline

Analyse :

L'outil natif pyATS confirme l'absence de dérive entre l'état opérationnel pré-déploiement et l'état post-déploiement sur les six Leafs Cisco. Les structures de données normalisées produites par les parsers Genie (interfaces, OSPF, VLANs, routes) sont strictement identiques dans les deux snapshots.

Arista EOS

L'étape POST-10b exécute `arista_diff.py` dans le container pyATS, qui compare terme à terme les deux fichiers JSON produits par `arista_audit.py` lors des phases PRE-5 et POST-9.

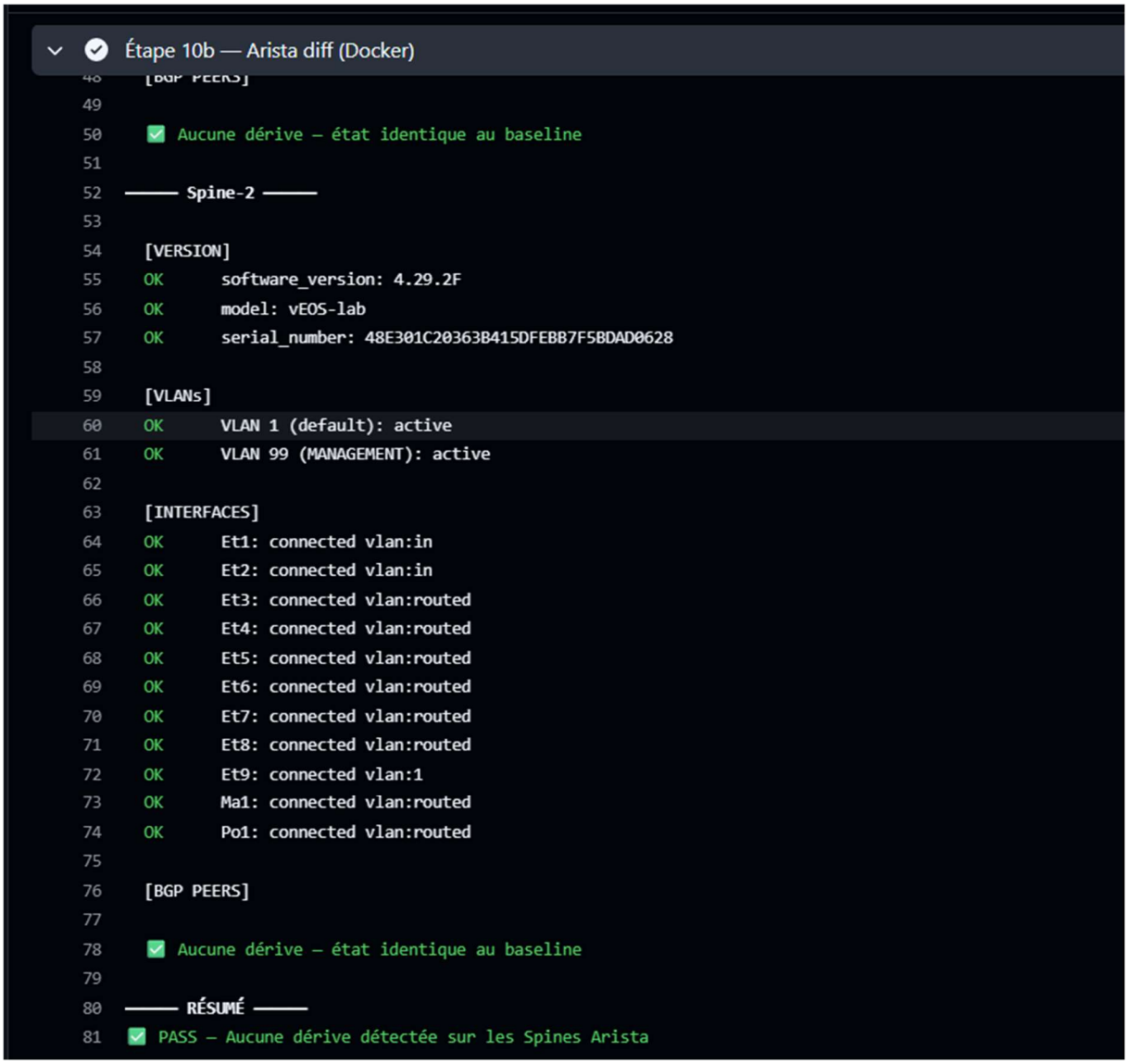


Figure 32 : Extrait de l'exécution de l'étape 10b dans le pipeleine

Analyse :

La comparaison porte sur les voisins OSPF et les interfaces des deux Spines. L'absence de dérive confirme que les configurations Arista EOS appliquées lors du déploiement sont strictement conformes aux données de la SSoT NetBox.

Validation du sprint 3

La baseline pré-déploiement est collectée sur les trois familles d'équipements. Les configurations ont été appliquées avec succès sur les Leafs Cisco IOS et les Spines Arista EOS. La convergence OSPF multi-plateforme est assurée.

5. Sprint 4 : Sécurisation périmétrique et validation de l'idempotence

Ce sprint couvre le déploiement du périmètre de sécurité FortiGate. Contrairement aux équipements réseau (Sprint 3), les FortiGates sont configurés exclusivement via API REST, sans établir de session SSH. Le sprint est organisé en deux volets : le déploiement proprement dit (§5.1) ainsi que la logique d'idempotence qui le gouverne (§5.2).

5.1 Déploiement FortiOS

Contrairement aux équipements réseau du Sprint 3, les FortiGates sont configurés exclusivement via API REST, sans établir de session SSH. Le playbook **`deploy_fortigates.yml`** délègue l'intégralité de la logique au rôle `firewall_fortinet` comme le montre le contenu de ce playbook ci-dessous :

```
- name: "FortiGate | Déploiement des configurations (API REST)"
  hosts: fortigates
  tasks:
    - import_role:
        name: firewall_fortinet
```

Le rôle `firewall_fortinet` est organisé en tâches fonctionnelles distinctes au run-88 : `system.yml`, `policies.yml`, `interfaces.yml`, `routing.yml`, `hardening.yml`, `dmz_vip.yml` et `observability.yml`. Chaque fichier de tâches couvre un périmètre de configuration indépendant et peut être exécuté séparément lors de mises à jour partielles.

5.2 Patterns d'idempotence et gestion des ressources

L'idempotence est garantie par deux patterns complémentaires :

1. Pattern A pour les ressources singleton : lecture de l'état courant, comparaison en mémoire, mise à jour conditionnelle ;
2. Pattern B pour les objets indexés : GET de l'état existant, POST si absent, PUT si différent.

L'extrait de la tâche `routing.yml` suivant illustre le Pattern B appliqué aux routes statiques :

```
- ansible.builtin.uri:
    url: "https://{{ ansible_host }}/api/v2/cmdb/router/static"
    method: GET
    headers: { Authorization: "Bearer {{ ansible_httpapi_token }}" }
  register: current_routes
  delegate_to: localhost

- ansible.builtin.uri:
    method: POST
    body: "{{ route }}"
  when: route['seq-num'] not in current_seq_nums
  delegate_to: localhost
```

Les routes proviennent de `host_vars/SD-WAN_NGFW.yml`, alimenté depuis NetBox via GraphQL par `netbox_sync.py`. Ce mécanisme garantit que chaque run du pipeline converge vers l'état déclaré dans la source de vérité, quel que soit l'état initial de l'équipement.

5.3 Résultat du déploiement FortiOS

Le rôle `firewall_fortinet` a effectué les lectures GET des huit ressources FortiGate sans détecter de dérive par rapport à la SSoT. Les tâches PUT correspondantes ont été sautées (skipped) en application du Pattern A d'idempotence. L'absence de tâche changed dans certaines taches confirme l’dempotence.

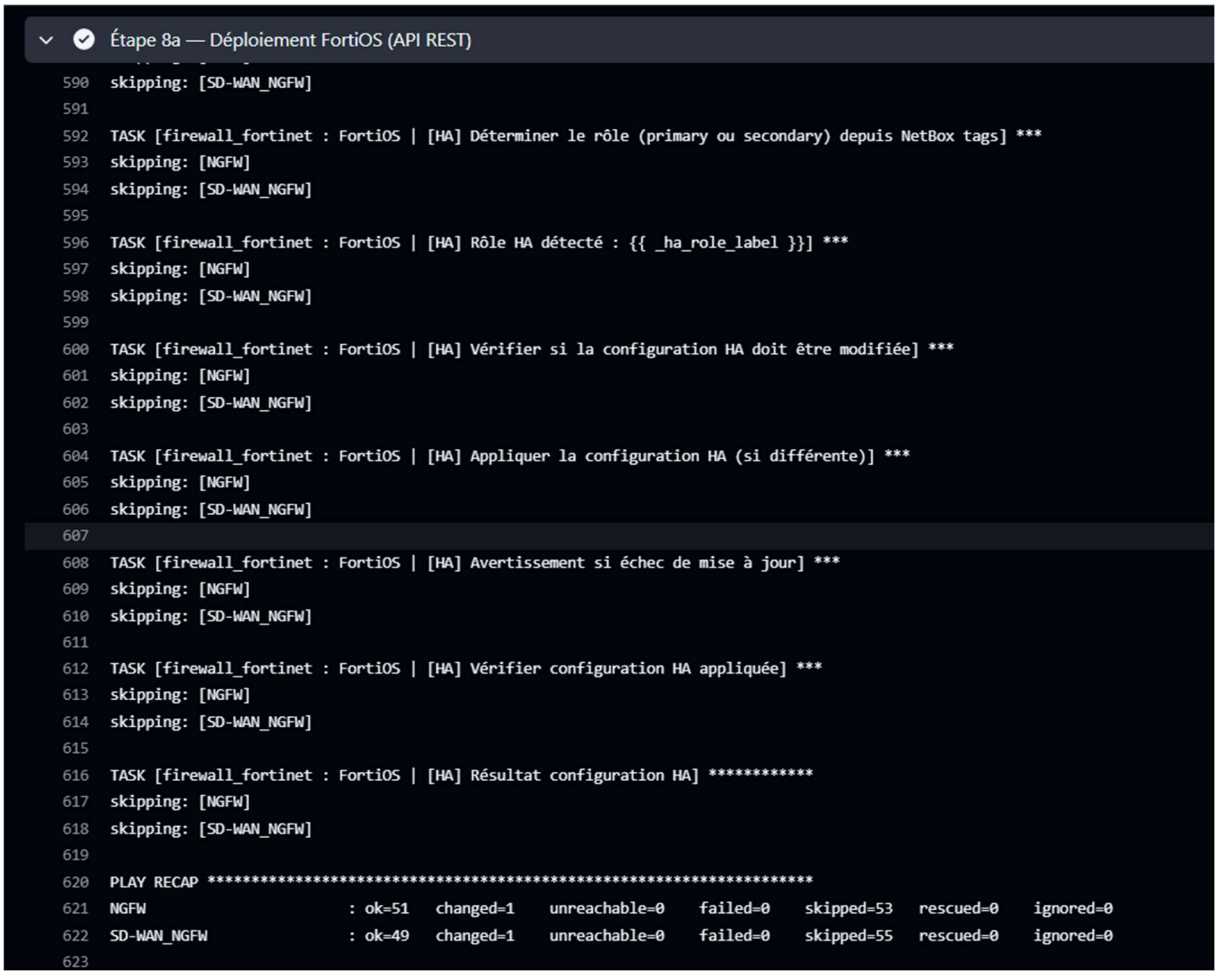


Figure 33 : Extrait de sortie de l'étape 8a du pipeline

5.4 Conformité post-déploiement FortiOS

L'étape POST-10c exécute `fortigate_diff.py`, qui compare les huit ressources collectées entre les phases PRE et POST. Cette étape est la seule à être strictement bloquante : tout écart FortiOS non résolu interrompt le pipeline immédiatement.

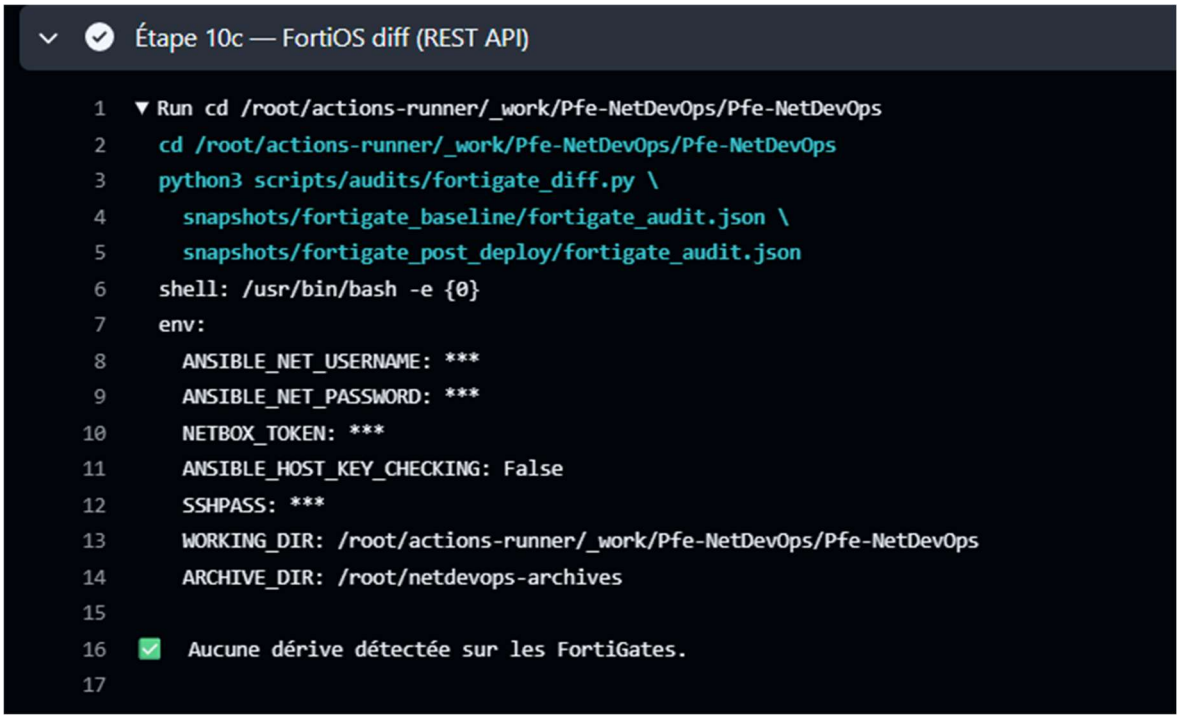


Figure 34 : Exécution de l'étape 10c du pipeline

Analyse :

Le rapport vide confirme que l'état opérationnel est strictement identique avant et après déploiement sur les deux FortiGates primaires. Aucune configuration parasite n'a été introduite par le pipeline. Ce résultat valide également la cohérence du rôle `firewall_fortinet` avec la source de vérité NetBox sur les huit ressources couvertes.

5.5 Idempotence globale

Un second run exécuté immédiatement après le run #88 ne produit aucune tâche changed dans les playbooks Ansible, sur aucune des trois familles d'équipements.

Analyse :

L'absence de modification lors d'une exécution successive confirme formellement que les playbooks sont idempotents. L'infrastructure reste stable quels que soient le nombre de passages du pipeline ou l'intervalle entre deux exécutions. Ce comportement est conforme à la propriété d'idempotence et constitue l'une des garanties fondamentales du paradigme NetDevOps mis en œuvre dans ce projet.

Validation du sprint 4

Le FortiGate est configuré via API REST de façon idempotente. Les politiques de sécurité, les routes statiques WAN et les interfaces sont en ligne avec la source de vérité NetBox. La convergence LACP/STP est assurée par la pause de 30 secondes.

6. Sprint 5 : Audit, conformité et traçabilité

Ce sprint clôture le pipeline par la vérification formelle de la conformité entre l'état exprimé dans NetBox et l'état observé sur les équipements (§6.1). L'archivage et la traçabilité (§6.2) ainsi que les contraintes techniques rencontrées au cours du projet sont documentées en §7.

6.1 Audit post-déploiement multi-constructeurs

La phase POST reproduit exactement les vecteurs de collecte de PRE-5, vers des répertoires de sortie distincts, garantissant la comparabilité stricte des deux états. La comparaison est effectuée par famille d'équipements :

- Cisco IOS (étape 10a)
- Arista EOS (étape 10b)
- FortiOS (étape 10c)

L'absence de ligne de dérive dans les trois rapports constitue la preuve formelle que le pipeline a produit l'état désiré.

6.2 Archivage et nettoyage

Exécutée inconditionnellement, l'étape 11 du pipeline archive l'ensemble des artefacts du run dans `/root/netdevops-archives/run-<N>/` sur le nœud KVM :

- Snapshots baseline et post-déploiement des trois familles d'équipements
- Configurations générées et host_vars/
- Fichier d'inventaire hosts
- Fichier sentinelle de validation Batfish
- Fiche de run horodatée, synthétisant le numéro de run, l'horodatage, la branche Git et le SHA du commit

Chaque run dispose d'une entrée d'archive indépendante, garantissant la traçabilité complète de chaque exécution et la possibilité de retour à tout état antérieur.

```bash
echo "Run       : ${{ github.run_number }}"          > run_info.txt
echo "Date      : $(date -u '+%Y-%m-%d %H:%M UTC')"  >> run_info.txt
echo "Commit    : ${{ github.sha }}"                 >> run_info.txt
echo "Déclenché : ${{ github.actor }}"               >> run_info.txt
```

Figure 35 : Extrait de l'étape de génération de la fiche de run à l'étape 11

Structure de l'archive du run #88 :

```
/root/netdevops-archives/run-88/
├── arista_baseline/          # État pré-déploiement Arista
├── arista_post_deploy/       # État post-déploiement Arista
├── baseline/                 # État pré-déploiement Cisco
├── fortigate_baseline/       # État pré-déploiement FortiGate
├── fortigate_post_deploy/    # État post-déploiement FortiGate
├── generated_configs/        # Configurations générées (20 fichiers)
├── host_vars/                # Variables d'hôte générées
├── hosts                     # Inventaire Ansible
├── post_deploy/              # État post-déploiement Cisco
├── run_info.txt              # Fiche de run horodatée
└── validation_passed         # Sentinelle Batfish
```

Fiche de run :

La fiche de run horodatée constitue le document de synthèse de chaque exécution :

```
Run : 88

Date : 2026-06-05 16:07 UTC
```

```
Commit : 8e40a72f2e7039dc886aa111a5a257b1e43abe11

Déclenché par : Mouh4x
```

En cas d'incident ou d'audit de conformité, la fiche de run permet de retrouver en un seul document l'ensemble des informations nécessaires à la reconstruction de l'état du réseau au moment de l'exécution. Chaque run dispose d'une entrée d'archive indépendante, assurant la traçabilité permanente.

L'étape 12, également inconditionnelle, supprime le fichier `.vault_pass` du workspace GitHub Actions, quelle que soit l'issue du pipeline.

Vue d'ensemble du pipeline #88 dans GitHub Actions :

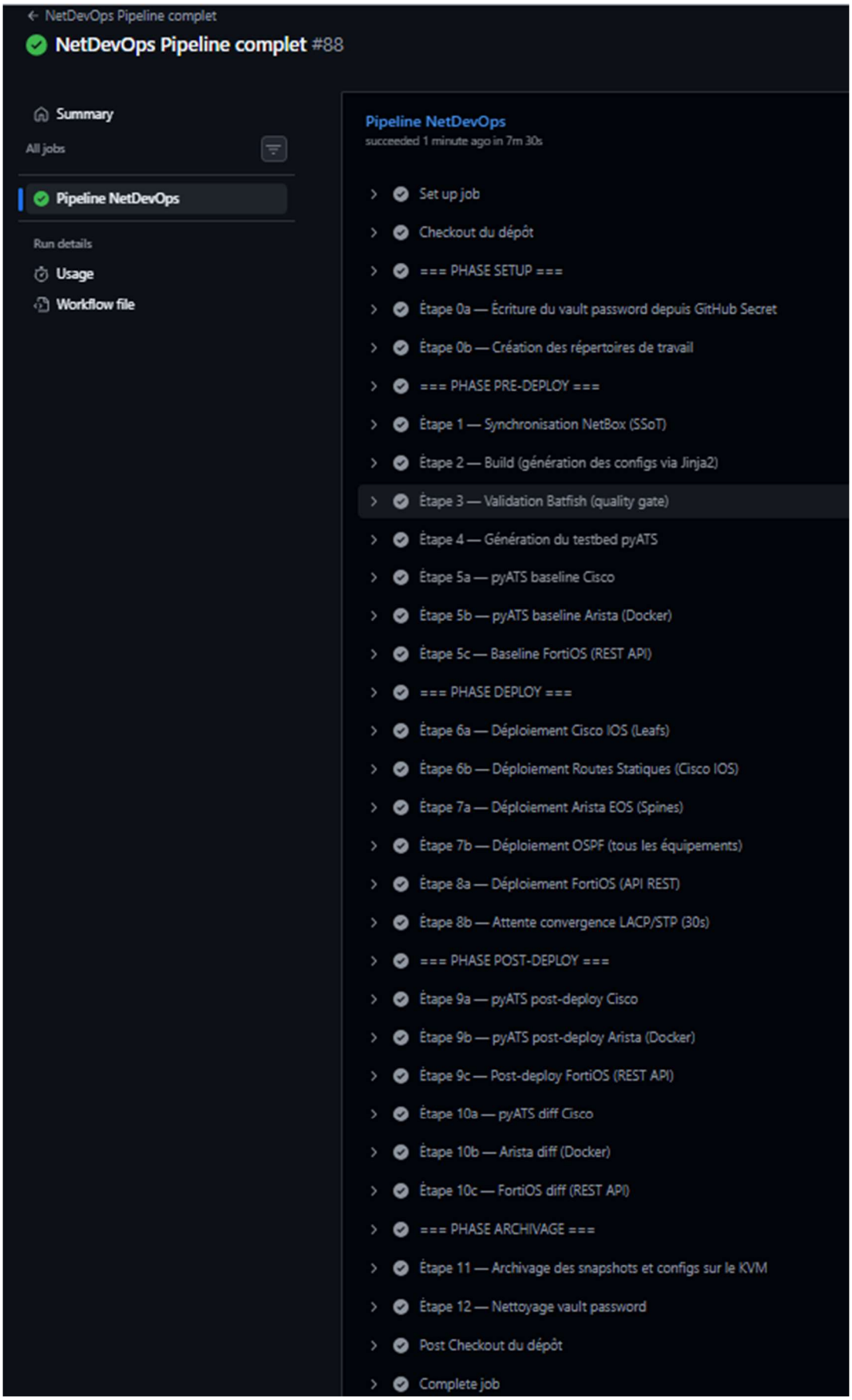


Figure 36 : Vue du pipeline #88 dans GitHub Actions

Les trois phases (PRE, DEPLOY, POST) se sont enchaînées sans erreur. L'enchaînement conditionnel des phases garantit l'intégrité du processus : chaque étape n'est déclenchée qu'à la condition que la précédente se soit terminée avec succès selon les conditions exprimées, rendant le pipeline auto-arrêtant en cas d'anomalie.

Validation du sprint 5

L'audit post-déploiement confirme la conformité entre l'état exprimé dans NetBox et l'état observé sur les équipements. L'absence de toute ligne de dérive dans les trois rapports constitue la preuve formelle que le pipeline a produit l'état désiré. Les artefacts du run sont archivés et traçables.

7. Contraintes techniques et ajustements

Incompatibilité de la collection fortinet.fortios :

La collection attendait des structures de données propres à des versions de FortiOS supérieures à celle disponible dans EVE-NG. Le rôle `firewall_fortinet` a été intégralement migré vers ansible.builtin.uri en API REST directe, produisant trois bénéfices : indépendance vis-à-vis de la version, contrôle explicite des codes HTTP et implémentation native des patterns d'idempotence. (Fortinet, Inc., s.d.)

Contraintes propres à l'environnement EVE-NG :

Trois incompatibilités ont nécessité des ajustements :

- ios_config avec pylibssh sur vIOS : comportement erratique lors du déploiement OSPF, résolu par délégation à configure_ospf_ios.py via Netmiko (Ansible Pylibssh Project, s.d.) ;
- Genie non disponible pour EOS : collecte rebasée sur device.execute() et parsing par expression régulière en container (cf. section 4.1) (Cisco Test Automation, s.d.) ;
- incompatibilité cryptographique de vEOS avec OpenSSH : testbed statique dédié `inventory/testbed_docker.yaml` maintenu en complément.

Abandon du plugin d'inventaire natif NetBox : le plugin netbox.netbox.nb_inventory a été abandonné au profit de `netbox_sync.py`, en raison de l'impossibilité d'accéder aux données OSPF et aux routes statiques via GraphQL depuis le plugin, et du besoin d'un contrôle total sur la transformation des données injectées dans les `host_vars/`. (NetBox Labs, s.d.)

Enseignements de l'implémentation : la qualité du pipeline est directement conditionnée par la rigueur de la saisie dans NetBox. Une adresse IP incorrecte ou un champ absent peuvent produire une configuration qui passe la validation Batfish tout en déployant un état erroné. Batfish et l'audit opérationnel sont complémentaires : Batfish détecte les anomalies de plan de contrôle avant déploiement, pyATS et les scripts d'audit mesurent la conformité de l'état réel avant et après déploiement. Aucun des deux seul ne suffit.

8. Conclusion

Ce chapitre a permis de mettre en œuvre et de valider, sprint après sprint, l'intégralité de la chaîne d'automatisation conçue au chapitre 2. L'exécution de bout en bout du pipeline (run #88) a démontré, sans intervention manuelle et sans dérive sur les trois familles d'équipements, que les objectifs de fiabilité, d'idempotence et de traçabilité sont effectivement atteints. Les ajustements rencontrés au cours de l'implémentation et les perspectives d'évolution identifiées ouvrent des pistes de renforcement de la robustesse opérationnelle du pipeline.

.

Conclusion générale et perspectives d'évolution

Ce mémoire a présenté la conception et la mise en œuvre d'une chaîne d'automatisation réseau complète, réalisée dans le cadre d'un stage de fin d'études au sein d'INEOS. Fondée sur le paradigme NetDevOps et déployée dans un environnement de simulation multi-constructeurs (Cisco IOS, Arista EOS, FortiOS), la solution démontre qu'il est possible de gérer des changements de configuration de manière entièrement automatisée, auditable et reproductible, une exigence d'autant plus critique pour des clients dont l'infrastructure conditionne la continuité de services financiers.

Les cinq sprints de réalisation ont permis d'atteindre l'ensemble des objectifs fixés au chapitre 1 : la source de vérité NetBox constitue le point d'entrée unique de toute modification de l'infrastructure ; la validation statique Batfish forme une barrière préventive avant tout déploiement effectif ; les patterns d'idempotence garantissent la stabilité du réseau lors d'exécutions successives ; et les rapports d'audit post-déploiement établissent la conformité formelle entre l'état déclaré dans la source de vérité et l'état observé sur les équipements. Le run #88, exécuté de bout en bout sans intervention manuelle, en constitue la preuve opérationnelle.

Perspectives d'évolution

Le pipeline d'automatisation mis en place pose les bases d'une infrastructure réseau agile et évolutive. Plusieurs axes de développement sont envisagés pour maximiser son impact.

À court terme : Consolidation et réactivité opérationnelle

- **Extension de Batfish :** Intégrer la validation des politiques FortiGate afin de couvrir l'intégralité du parc d'équipements au niveau de la *quality gate* pré-déploiement.
- **Supervision POST active :** Coupler Event-Driven Ansible au pipeline pour déclencher des alertes automatiques immédiates en cas de dérive de configuration post-déploiement.

À moyen terme : Autonomie et *Self-Healing*

- **Zero-Touch Provisioning (ZTP) :** Automatiser le remplacement du matériel défaillant. Le nouvel équipement devra s'auto-configurer de manière autonome en interrogeant NetBox et en exécutant les playbooks Ansible dédiés, sans aucune intervention humaine sur la CLI.
- **Transformation métier :** Faire évoluer le rôle de l'équipe réseau de l'exécution de tâches répétitives vers la supervision d'un système autonome.

À long terme : Intelligence Artificielle (IA) et Standardisation

- **Analyse prédictive (IA + pyATS) :** Intégrer un modèle de langage (LLM) connecté à pyATS et NetBox pour analyser les logs, identifier les anomalies comportementales et soumettre des plans d'action structurés à validation humaine.
- **Intent-Based Networking :** Configurer le réseau par langage naturel (ex: *"Déployer un nouveau VLAN de gestion sur les Leafs"*). L'IA traduirait cette consigne en modèles NetBox et playbooks Ansible, validés en amont par Batfish.

ANNEXES

ANNEXE A : Infrastructure de simulation et configuration Docker

Cette annexe regroupe les éléments de fondation technique du projet : la personnalisation de l'image NetBox, l'orchestration Docker Compose (section A.1) et la configuration de référence du routeur ISP-1 (section A.2).

A.1 Personnalisation de l'image NetBox

La personnalisation de l'image NetBox, nécessaire à l'intégration du plugin netbox-routing évoquée en §2.2 (Infrastructure Docker), repose sur deux fichiers versionnés dans le dépôt et reproduits intégralement ci-après. Le premier, `Dockerfile-Plugins`, étend l'image officielle netboxcommunity/netbox:latest par l'installation du plugin netbox-routing, déclaré dans `plugin_requirements.txt` :

```
root@kvm:~/netbox-docker# cat Dockerfile-Plugins
FROM netboxcommunity/netbox:latest
COPY ./plugin_requirements.txt /opt/netbox/
RUN /usr/local/bin/uv pip install -r /opt/netbox/plugin_requirements.txt
root@kvm:~/netbox-docker# cat plugin_requirements.txt
netbox-routing
root@kvm:~/netbox-docker#
```

Figure 37 : Fichier Dockerfile-Plugins

Le second fichier, `docker-compose.override.yml`, applique l'image résultante, netbox-custom:latest, aux trois services netbox, netbox-worker et netbox-housekeeping, et expose l'interface Web sur le port 8000 :

```
root@kvm:~/netbox-docker# cat docker-compose.override.yml
services:
  netbox:
    build:
      context: .
      dockerfile: Dockerfile-Plugins
    image: netbox-custom:latest
    ports:
      - "8000:8080"
  netbox-worker:
    image: netbox-custom:latest
  netbox-housekeeping:
    image: netbox-custom:latest
root@kvm:~/netbox-docker#
```

Figure 38 : Ficher docker-compose.override.yml

Ces deux fichiers constituent l'intégralité de la personnalisation référencée en §2.2 ; le fichier `docker-compose.yml` d'origine demeure quant à lui inchangé, conformément au principe d'isolation des surcharges retenu pour le projet.

A.2 Configuration de référence (routeur ISP-1)

Conformément au renvoi de §2.3 (Topologie EVE-NG et connectivité), la configuration complète de ISP-1 est reproduite ci-dessous à titre de référence de l'environnement WAN simulé. Elle détaille la déclaration des deux liaisons vers SD-WAN_NGFW (Ethernet0/0 et Ethernet0/2), la translation d'adresses appliquée en sortie sur Ethernet0/1, ainsi que la route par défaut vers la passerelle 10.66.14.1 :

```
ISP-1#
ISP-1#show running-config | section interface
interface Ethernet0/0
 description LIEN-PRINCIPAL-SDWAN-PORT3
 ip address 10.200.1.1 255.255.255.252
 ip nat inside
 ip virtual-reassembly in
interface Ethernet0/1
 description UPLINK-REAL-INTERNET-PNET0
 ip address dhcp
 ip nat outside
 ip virtual-reassembly in
interface Ethernet0/2
 description LIEN-REDONDANT-SDWAN-PORT5
 ip address 10.200.2.1 255.255.255.252
 ip nat inside
 ip virtual-reassembly in
interface Ethernet0/3
 no ip address
 shutdown
ip nat inside source list NAT_ACL interface Ethernet0/1 overload
ISP-1#show running-config | section ip nat
 ip nat inside
 ip nat outside
 ip nat inside
ip nat inside source list NAT_ACL interface Ethernet0/1 overload
ISP-1#show running-config | section ip route
ip route 0.0.0.0 0.0.0.0 10.66.14.1
ISP-1#
```

Figure 39 : Configuration de ISP-1

Cette configuration reste statique sur la durée du projet : ISP-1 est volontairement maintenu hors du périmètre Ansible et n'assure que le rôle de fournisseur d'accès, conformément au choix architectural exposé en §2.3.

ANNEXE B : Extraits sélectionnés des scripts d'automatisation

Les extraits suivants constituent les détails d'implémentation des scripts d'automatisation cités dans le corps du rapport aux sections §3.2 et §4.1 (chapitre 2).

B.1 Structure du workflow CI/CD

Conformément au renvoi de §3.1, l'extrait suivant présente les lignes de détail condensée du fichier de workflow `netdevops-pipeline.yml` :

```yaml
# ──────────────────────────────────────────────────────────────────────
# Annexe B — Extrait annoté : .github/workflows/pipeline.yml
# Pipeline NetDevOps — 4 phases (SETUP · PRE · DEPLOY · POST)
# ──────────────────────────────────────────────────────────────────────

name: "NetDevOps Pipeline complet"

# ── DÉCLENCHEURS ──────────────────────────────────────────────────────
on:
  workflow_dispatch:                  # (1) déclenchement manuel ou re-run
  push:
    branches: ["main"]
    paths:                            # (2) push ciblé : seuls les fichiers
      - "roles/**/templates/**"       #     métier déclenchent le pipeline ;
      - "scripts/netbox_sync.py"      #     docs, tests unitaires en sont
exclus
      - "playbooks/*.yml"             #     (liste complète dans le dépôt)
      # ...
  repository_dispatch:
    types: [netbox-webhook]           # (3) webhook NetBox → tout changement
SSoT

# Un seul run actif par branche à tout instant.
# cancel-in-progress: false empêche l'interruption d'un déploiement en cours.
concurrency:
  group: netdevops-pipeline-${{ github.ref }}
  cancel-in-progress: false

jobs:
  pipeline:
    runs-on: self-hosted              # runner sur le KVM (accès réseau
interne)
    env:
      # Les secrets GitHub sont injectés en variables d'env, jamais en clair.
      ANSIBLE_NET_USERNAME: ${{ secrets.DEVICE_USERNAME }}
```

```
      ANSIBLE_NET_PASSWORD: ${{ secrets.DEVICE_PASSWORD }}
      NETBOX_TOKEN:         ${{ secrets.NETBOX_TOKEN }}
      ANSIBLE_HOST_KEY_CHECKING: "False"
      ARCHIVE_DIR: /root/netdevops-archives  # persistance hors workspace Git

    steps:

# ── PHASE SETUP ──────────────────────────────────────────────────
      - name: "0a — Écriture du vault password"
        run: |
          # Matérialisé en fichier temporaire (chmod 600), référencé par
          # ansible.cfg comme --vault-password-file. Supprimé à l'étape 12.
          echo "${{ secrets.VAULT_PASS }}" > .vault_pass && chmod 600
.vault_pass

      - name: "0b — Création des répertoires de travail"
        run: |
          mkdir -p build/{generated_configs,batfish_snapshot/configs}
          mkdir -p snapshots/{baseline,post_deploy}
          mkdir -p snapshots/{arista_baseline,arista_post_deploy}
          mkdir -p snapshots/{fortigate_baseline,fortigate_post_deploy}

# ── PHASE PRE-DEPLOY ─────────────────────────────────────────────
      - name: "1 — Synchronisation NetBox (SSoT)"
        run: python3 scripts/netbox_sync.py
        # → génère host_vars/<device>.yml + hosts depuis l'API NetBox/GraphQL.

      - name: "2 — Build Jinja2 (génération des configs)"
        run: ansible-playbook -i hosts playbooks/build_network.yml
        # → produit build/generated_configs/*.cfg ; connection: local, aucun
équipement contacté.

      - name: "3 — Validation Batfish (quality gate)"
        run: python3 scripts/validate_batfish.py
        # → crée .validation_passed si OK ; les playbooks DEPLOY le vérifient
en gate.

      - name: "4 — Génération testbed pyATS"
        run: python3 scripts/generate_pyats_testbed.py
```

```
      # Étape 5 : 3 sous-étapes — une par famille OS (vecteurs de collecte
distincts)

      - name: "5a — Baseline Cisco IOS (pyATS natif)"

        run: pyats learn platform interface --testbed-file
inventory/testbed_cisco.yaml --output snapshots/baseline

        continue-on-error: true         # non-bloquant : un réseau
partiellement UP reste déployable


      - name: "5b — Baseline Arista EOS (container pyATS)"

        run: |

          # vEOS est incompatible avec OpenSSH du runner hôte → exécution
déléguée au container.

          docker run --rm --network host \

            -v inventory/testbed_docker.yaml:/testbed.yaml:ro \

            -v scripts/audits/arista_audit.py:/arista_audit.py:ro \

            -v snapshots/arista_baseline:/output \

            ciscotestautomation/pyats:latest-full bash -c "python3
/arista_audit.py /output"


      - name: "5c — Baseline FortiOS (API REST)"

        run: python3 scripts/audits/fortigate_audit.py
snapshots/fortigate_baseline

        # → auto-découverte des FortiGates via host_vars/ (présence de
ansible_httpapi_token).


# ── PHASE DEPLOY ──────────────────────────────────────────────────

      - { name: "6  — Cisco IOS (Leafs)",      run: "ansible-playbook -i hosts
playbooks/deploy_network.yml"  }

      - { name: "6b — Routes statiques IOS",   run: "ansible-playbook -i hosts
playbooks/deploy_routes.yml"   }

      - { name: "7  — Arista EOS (Spines)",    run: "ansible-playbook -i hosts
playbooks/deploy_spines.yml"   }

      - { name: "7b — OSPF multi-plateforme",  run: "ansible-playbook -i hosts
playbooks/deploy_ospf.yml"     }

      - { name: "8a — FortiOS (API REST)",     run: "ansible-playbook -i hosts
playbooks/deploy_fortigates.yml" }

      - { name: "8b — Convergence LACP/STP",   run: "sleep 30"
}

      # sleep 30 : les Spines Arista ont besoin de ~20 s pour reconverger le
LACP


# ── PHASE POST-DEPLOY ─────────────────────────────────────────────

      # Étapes 9a/9b/9c : miroir exact de PRE-5 vers snapshots/*_post_deploy/

      # (même structure, répertoires de sortie distincts — non reproduites ici)
```

```
      - name: "10a — Diff Cisco (pyATS natif)"
        run: pyats diff snapshots/baseline snapshots/post_deploy || true
        continue-on-error: true         # non-bloquant : dérive Cisco = alerte,
pas arrêt


      - name: "10b — Diff Arista EOS (container)"
        run: |
          docker run --rm \
            -v scripts/audits/arista_diff.py:/arista_diff.py:ro \
            -v snapshots/arista_baseline/arista_audit.json:/baseline.json:ro \
            -v snapshots/arista_post_deploy/arista_audit.json:/post.json:ro \
            ciscotestautomation/pyats:latest-full bash -c "python3
/arista_diff.py /baseline.json /post.json"


      - name: "10c — Diff FortiOS   [BLOQUANT]"
        run: |
          python3 scripts/audits/fortigate_diff.py \
            snapshots/fortigate_baseline/fortigate_audit.json \
            snapshots/fortigate_post_deploy/fortigate_audit.json
        # Seule étape POST strictement bloquante : toute dérive FortiOS
        # non résolue provoque un exit code != 0 et interrompt le pipeline.


# ── ARCHIVAGE & NETTOYAGE ──────────────────────────────────────────
      - name: "11 — Archivage sur le KVM"
        if: always()                    # exécuté même en cas d'échec du
pipeline
        run: |
          RUN_DIR="${{ env.ARCHIVE_DIR }}/run-${{ github.run_number }}"
          mkdir -p "$RUN_DIR"
          # Artefacts copiés : snapshots (baseline + post_deploy pour les 3
OS),
          # configs générées, host_vars, inventaire, sentinelle Batfish.
          for src in snapshots/baseline snapshots/post_deploy \
                     snapshots/arista_baseline snapshots/arista_post_deploy \
                     snapshots/fortigate_baseline
snapshots/fortigate_post_deploy \
                     build/generated_configs host_vars; do
            [ -e "$src" ] && cp -r "$src" "$RUN_DIR/" || true
          done
          cp hosts "$RUN_DIR/hosts" 2>/dev/null || true
          cp build/batfish_snapshot/.validation_passed
"$RUN_DIR/validation_passed" 2>/dev/null || true
```

```
          # Fiche de run horodatée (référence Git + acteur + timestamp)
          printf "Run       : ${{ github.run_number }}\n"     >
"$RUN_DIR/run_info.txt"
          printf "Date      : $(date -u '+%Y-%m-%d %H:%M UTC')\n" >>
"$RUN_DIR/run_info.txt"
          printf "Commit    : ${{ github.sha }}\n"            >>
"$RUN_DIR/run_info.txt"
          printf "Déclenché : ${{ github.actor }}\n"          >>
"$RUN_DIR/run_info.txt"


      - name: "12 — Suppression du vault password"
        if: always()
        run: rm -f .vault_pass          # supprimé inconditionnellement, même
en cas d'échec
```

Références

Ansible Pylibssh Project. (n.d.). *Issue #549: pylibssh unable to handle a long stream of output from command execution*. (Ansible Pylibssh Project) Retrieved from https://github.com/ansible/pylibssh/issues/549

Cisco Systems, Inc. (2026). *pyATS and Genie Open Source Documentation*. (Cisco DevNet) Retrieved Mars 15, 2026, from https://developer.cisco.com/docs/pyats/

Cisco Test Automation. (n.d.). *Genie Parsers Source Repository and Supported Platforms*. Retrieved Juin 02, 2026, from https://github.com/CiscoTestAutomation/genieparser

Docker, Inc. (2026). *Docker Compose Overview and Specification*. (Docker Documentation) Retrieved Avril 03, 2026, from https://docs.docker.com/compose/

EVE-NG LTD. (2023, Mai). *EVE-NG Professional Cookbook*. (EVE-NG LTD) Retrieved Mars 04, 2026, from https://www.eve-ng.net/wp-content/uploads/2023/05/EVE-PE-BOOK-5.9-2023.pdf

Fogel, A., Fung, S., & Pedrosa, L. (2015). *A General Approach to Network Configuration Analysis (Batfish)*. (ACM SIGCOMM / Microsoft Research) Retrieved Mars 10, 2026, from https://github.com/batfish/batfish

Fortinet, Inc. (n.d.). *Ansible Galaxy Fortios Collection Documentation & Release Notes*. Retrieved Mai 13, 2026, from https://ansible-galaxy-fortios-docs.readthedocs.io/

GitHub, Inc. (2026). *GitHub Actions Documentation: About self-hosted runners*. (GitHub Docs) Retrieved Mars 15, 2026, from https://docs.github.com/fr/actions/hosting-your-own-runners/about-self-hosted-runners

INEOS Tech Forward. (n.d.). *Qui sommes-nous ?* (INEOS) Retrieved Fevrier 18, 2026, from INEOS TECH FORWORD: https://ineos.ma/qui-nous-sommes/

NetBox Labs. (2026). *NetBox Documentation: Network Source of Truth*. (NetBox Labs) Retrieved Mars 2026, from https://docs.netbox.dev/

NetBox Labs. (n.d.). *NetBox REST and GraphQL API Developer Guide*. Retrieved Mai 11, 2026, from https://docs.netbox.dev/en/stable/development/rpc-api/

Red Hat, Inc. (2026). *Ansible for Network Automation Documentation*. (Red Hat, Inc.) Retrieved Mars 06, 2026, from https://docs.ansible.com/ansible/latest/network/index.html